UNIT- II
Machine to Machine (M2M) and Embedded Programming for IoT

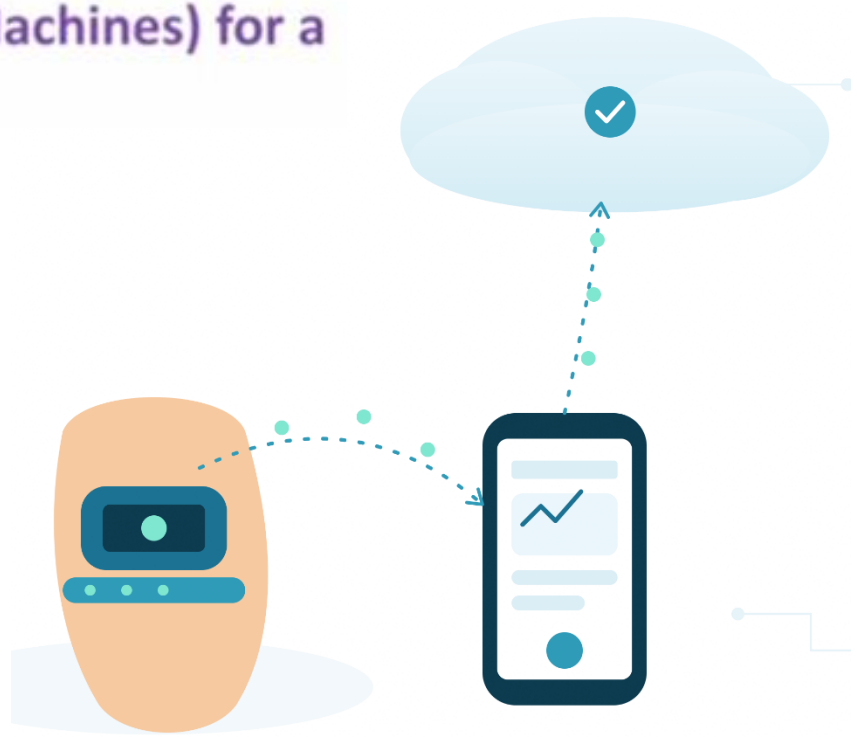
- **Machine-to-Machine Communications Overview**
- **Difference between IoT and M2M**
- **Interoperability in IoT: Standards and Protocols**
- **Introduction to Arduino Programming for IoT**
- **Integration of Sensors and Actuators with Arduino**
- **Hands-on Exercises:**
  - **Sensor Data Acquisition and Actuator Control**
  - **Basic Communication Protocols Implementation on Arduino**
- **IoT Device Interoperability Challenges and Solutions**

M2M Communications-Overview

- **M2M –** M2M provides direct communication between individual machines or devices. It is designed to communicate between devices (Machines) for a specific purpose.

Machines conversing

Devices exchanging data directly, with little or no human intervention.



Data reaching the cloud

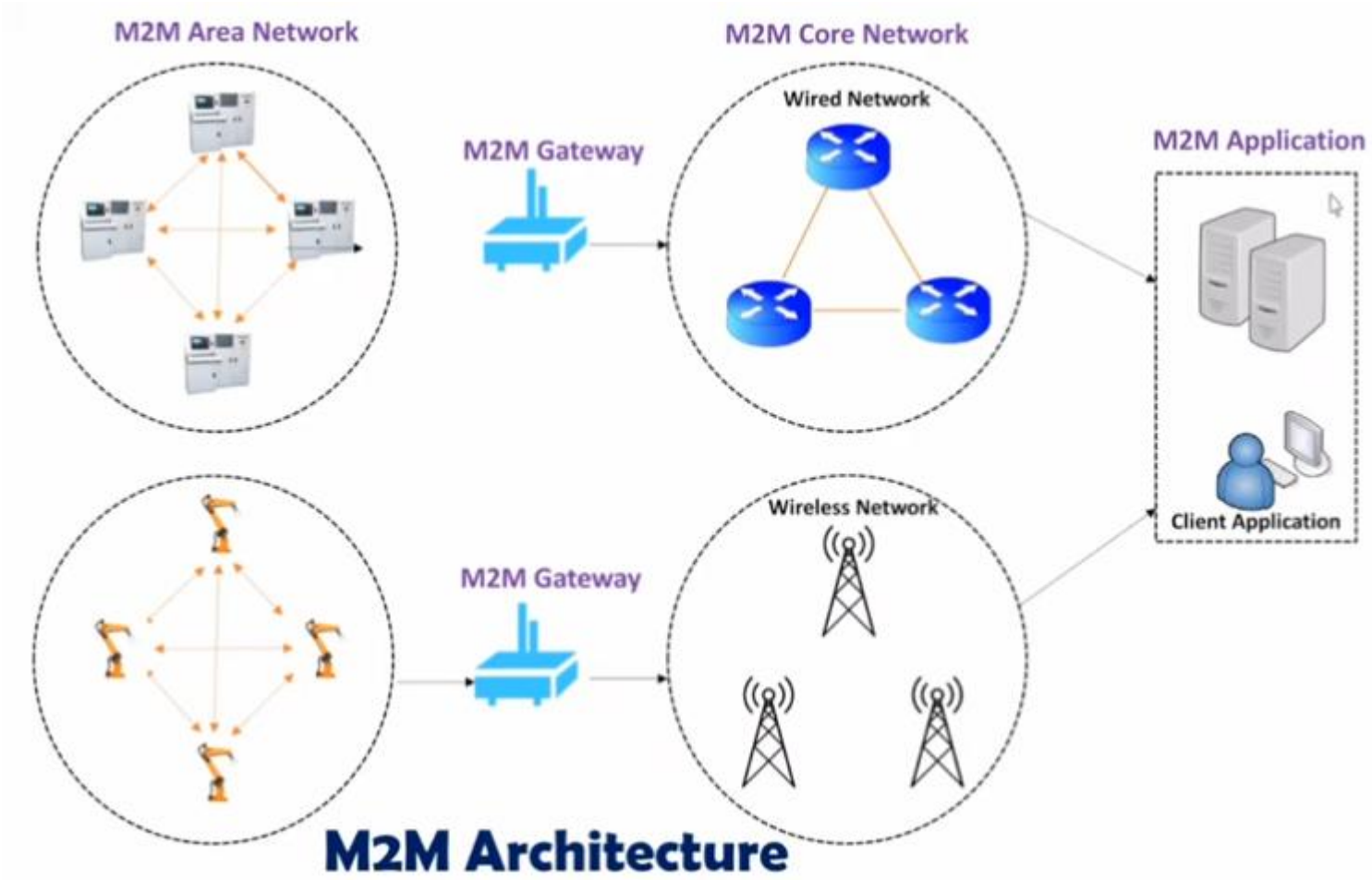
Information is collected, transmitted, and stored for processing.

Smart decisions

Systems sense, analyse, and act — automatically and in real time.

M2M Communications-Overview



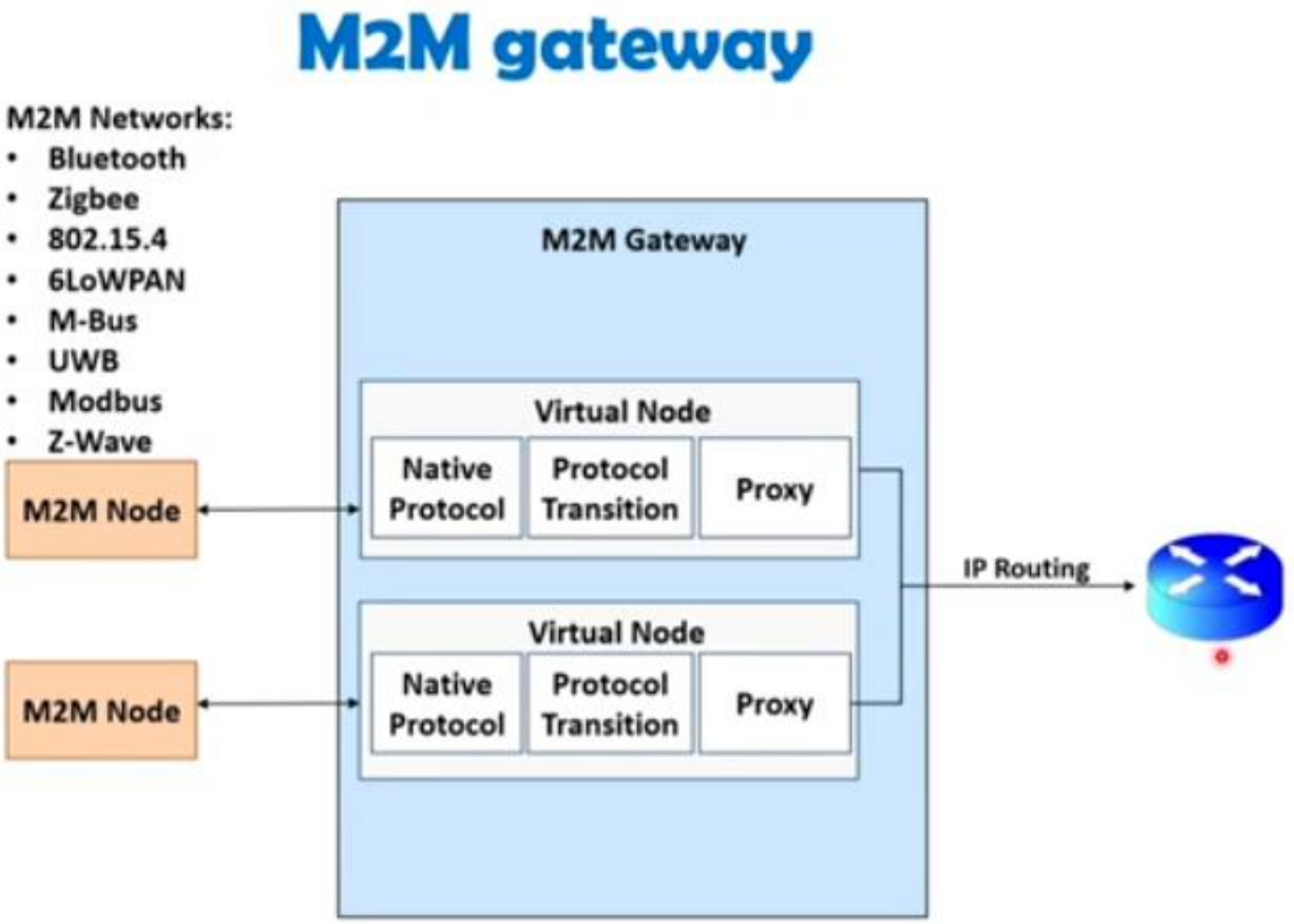


M2M Communications-Overview

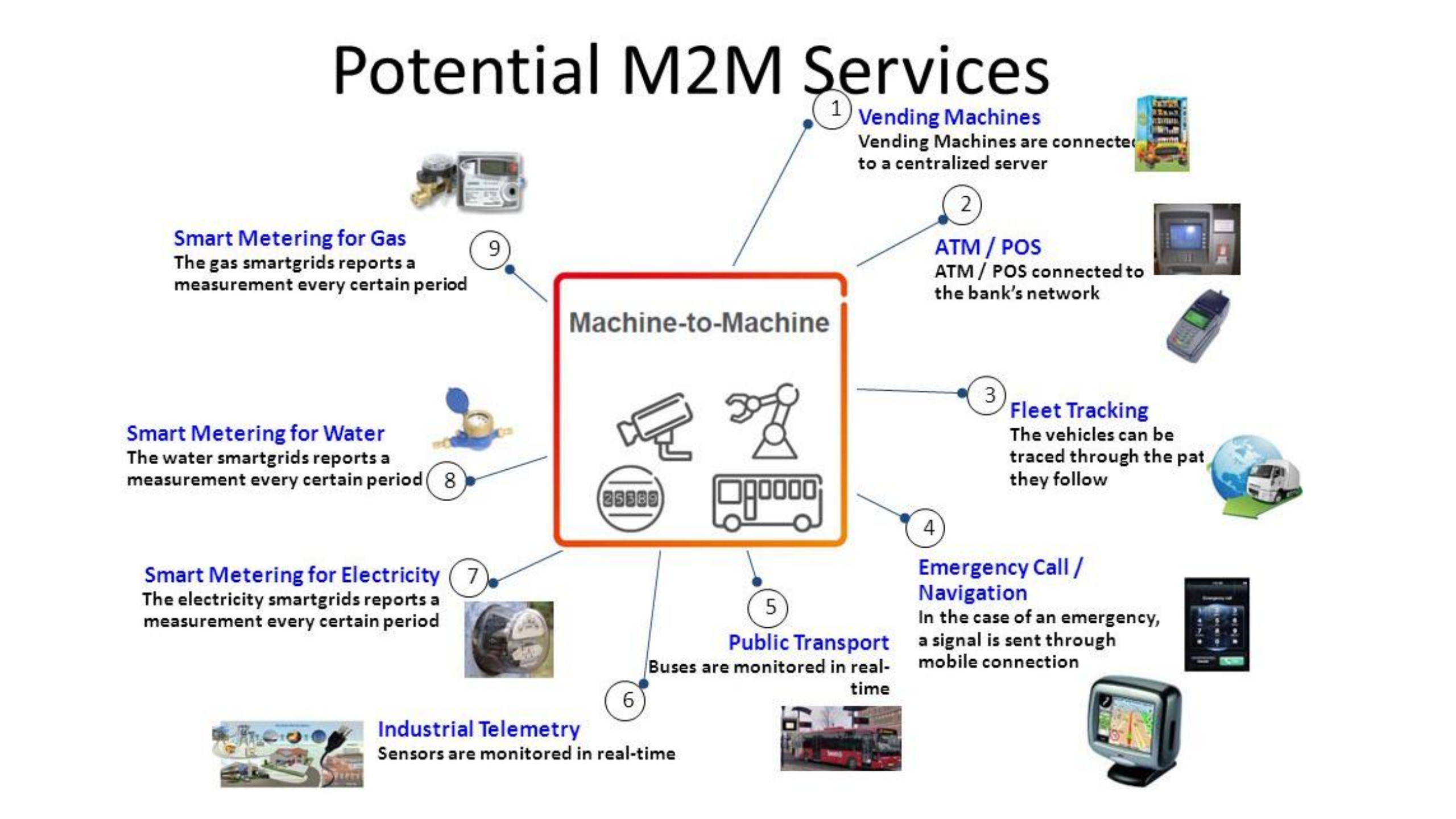
- **What is M2M gateway?**
- M2M gateway acts as an intermediatory between various devices and systems in a machine-to-machine setup.
- It provides the following functionalities:
  - Connectivity between machines.
  - Data exchange between machines.
  - Compatibilities between different networks.

- **What is the purpose or necessity of an M2M gateway?**
- Within the network nodes communicate with each other.
- To communicate with remote M2M area network, M2M gateway is required.

M2M Communications-Overview



Potential M2M Services



Differences between IoT and M2M

Similarities

- Both involve **machines/devices communicating with each other** without constant human intervention.
- Both use **sensors and communication technology** to collect and transmit data.
- Both aim to **automate processes** and enable data-driven decision-making.
- Both can use similar underlying communication protocols (cellular, Wi-Fi, Zigbee, etc.)
- IoT is often considered an **evolution/superset of M2M** — they share the same foundational concept of machines exchanging data.

Differences between IoT and M2M

Differences

Aspect	M2M	IoT
Definition	Direct communication between two machines/devices, typically point-to-point	Network of interconnected devices communicating over the internet, often through cloud platforms
Communication type	Point-to-point (device to device), often via proprietary or hardwired links	Device to cloud, device to device, and device to gateway — networked and internet-based
Internet dependency	Not necessarily internet-based — can use direct links, cellular, or private networks	Relies heavily on internet connectivity (IP-based networking)

Differences between IoT and M2M

Differences

Aspect	M2M	IoT
Scale	Usually limited/isolated systems (e.g., a vending machine reporting stock to a central server)	Massive scale — millions of interconnected devices globally
Data sharing	Data is usually used by a single application or a closed system	Data can be shared and accessed by multiple applications/stakeholders via the cloud
Protocols used	SNMP, proprietary protocols, cellular (GSM/GPRS)	HTTP, MQTT, CoAP, WebSocket, and other internet-standard protocols

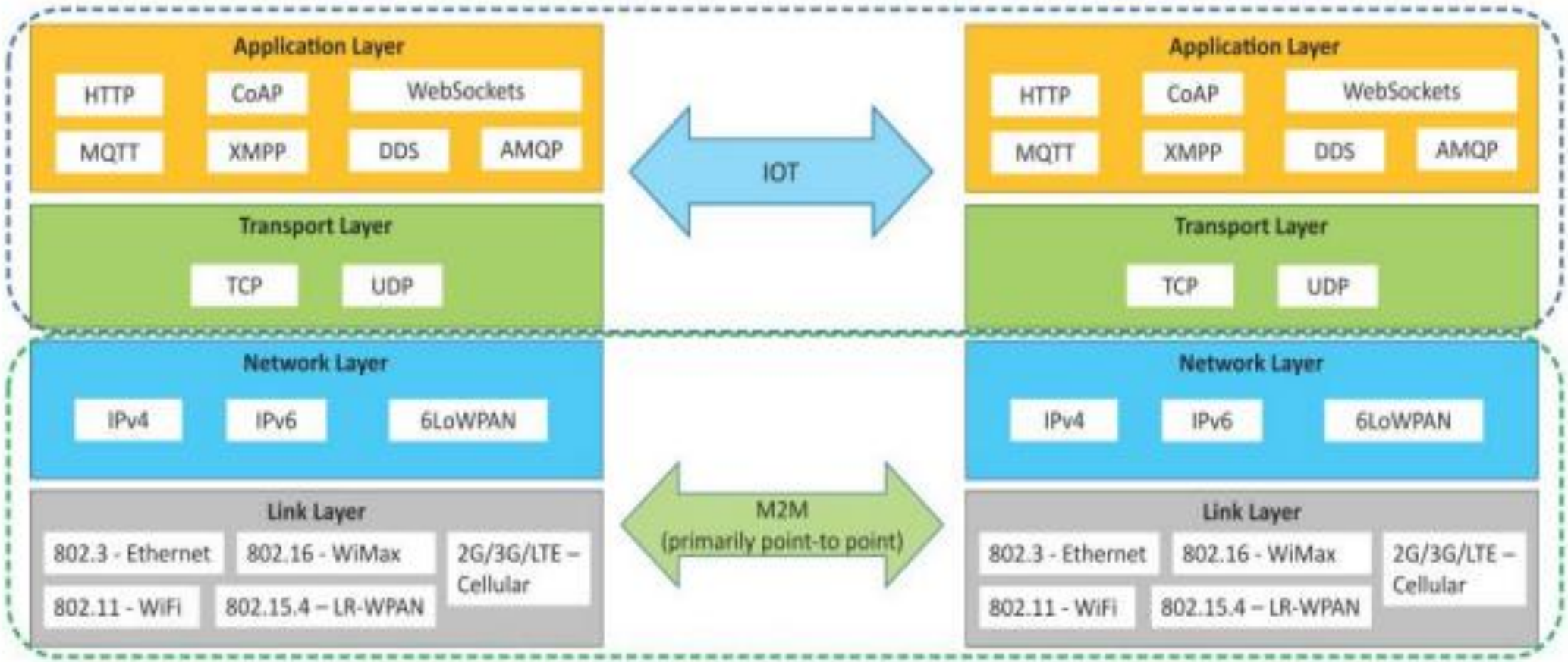
Differences between IoT and M2M

Differences

Aspect	M2M	IoT
Analytics/intelligence	Limited — mainly data transfer, minimal processing at the machine level	Strong emphasis on cloud-based analytics, AI/ML, big data processing
Human interaction	Minimal to none — machines act independently	Often includes human interfaces (apps, dashboards) for monitoring/control
Examples	ATM reporting cash levels to bank server, vending machine inventory reporting	Smart home systems, wearable fitness trackers, smart city infrastructure, connected cars
Interoperability	Limited — devices often communicate within a specific closed system	High — designed to work across various platforms and device types

Differences between IoT and M2M

Communication in IoT vs M2M



Interoperability in IoT: Standards and Protocols

How diverse devices, networks, and platforms learn to speak a common language

01 Why interoperability matters in IoT

02 Levels of interoperability

03 Communication & network protocols

04 Standards bodies and key standards

05 Approaches, case study & future trends

What Is IoT Interoperability?

Definition

The ability of heterogeneous IoT devices, networks, and platforms — often from different manufacturers — to exchange, interpret, and act on data correctly, without custom one-off integration.

In short: plug-and-play across brands, not just within one.

Devices

Sensors, actuators & gateways from many vendors

Data

Shared meaning, not just shared bytes

Platforms

Cloud, edge & apps working as one system

A Fragmented IoT Landscape

25B+

connected devices worldwide today

700+

distinct IoT protocols & standards in use

60%

of integration effort often spent on compatibility

Devices built for one ecosystem often cannot talk to devices from another — different transport protocols, data formats, and security models create silos instead of a single connected system.

Four Levels of Interoperability

1 Technical

Physical & network connectivity between devices (radios, cables, transport)

2 Syntactic

Common data formats & message structures (JSON, XML, CBOR)

3 Semantic

Shared meaning of exchanged data (ontologies, data models)

4 Organizational

Aligned business processes & policies across organizations

APPLICATION LAYER

Key Communication Protocols

M	**MQTT**	Lightweight publish/subscribe over TCP — ideal for constrained, low-bandwidth devices
C	**CoAP**	RESTful protocol over UDP built for resource-constrained sensor networks
A	**AMQP**	Reliable, queued messaging with strong delivery guarantees for enterprise IoT
H	**HTTP/S**	Familiar web protocol; heavier, but universally supported by web platforms

NETWORK LAYER

Connectivity Protocols

Wi-Fi

High throughput, short range

Bluetooth LE

Low power, personal-area range

Zigbee

Mesh networking for smart homes

Z-Wave

Low-power home automation mesh

LoRaWAN

Long range, very low power (WAN)

Thread

IP-based, mesh, low-power (Matter)

GOVERNANCE

Standardization Bodies

IEEE

Physical & link-layer standards (802.15.4, 802.11)

IETF

Internet protocols (6LoWPAN, CoAP, RPL)

W3C WoT

Web of Things — semantic interoperability

oneM2M

Common horizontal M2M/IoT service layer

OCF

Open Connectivity Foundation — device discovery & control

CSA

Connectivity Standards Alliance — the Matter standard

COMPARE

MQTT vs. CoAP vs. AMQP

	MQTT	CoAP	AMQP
Transport	TCP	UDP	TCP
Model	Publish / Subscribe	Request / Response	Queue-based messaging
Overhead	Very low	Very low	Higher
Best for	Telemetry, mobile	Sensor networks	Enterprise systems

SOLUTIONS

Approaches to Achieve Interoperability

Middleware & Gateways

Translate between protocols at the edge so devices never need to speak the same language natively

Semantic Web / Ontologies

Shared vocabularies (e.g., SAREF, SSN) give data consistent meaning across platforms

Unified APIs

Standardized REST/GraphQL interfaces abstract away underlying device protocols

Common Data Models

Shared schemas ensure sensors and apps interpret payloads identically

CASE STUDY

Matter: A Unifying Smart-Home Standard

Backed by the Connectivity Standards Alliance (Apple, Google, Amazon, Samsung SmartThings and others), Matter runs over Wi-Fi, Thread, and Bluetooth LE to let smart-home devices from different brands work together — and with any compatible voice assistant — out of the box.

A practical example of technical + organizational interoperability working together.

Transport

Wi-Fi · Thread · BLE

Layer unified

Application layer

Governed by

Connectivity Standards Alliance

REALITY CHECK

Challenges That Remain

Legacy devices

Millions of deployed devices predate modern standards and can't be updated

Security & trust

Bridging protocols must not create new attack surfaces

Standard fragmentation

Competing standards still coexist, slowing convergence

Scalability

Billions of devices strain discovery, addressing, and semantic matching

Where Interoperability Is Heading

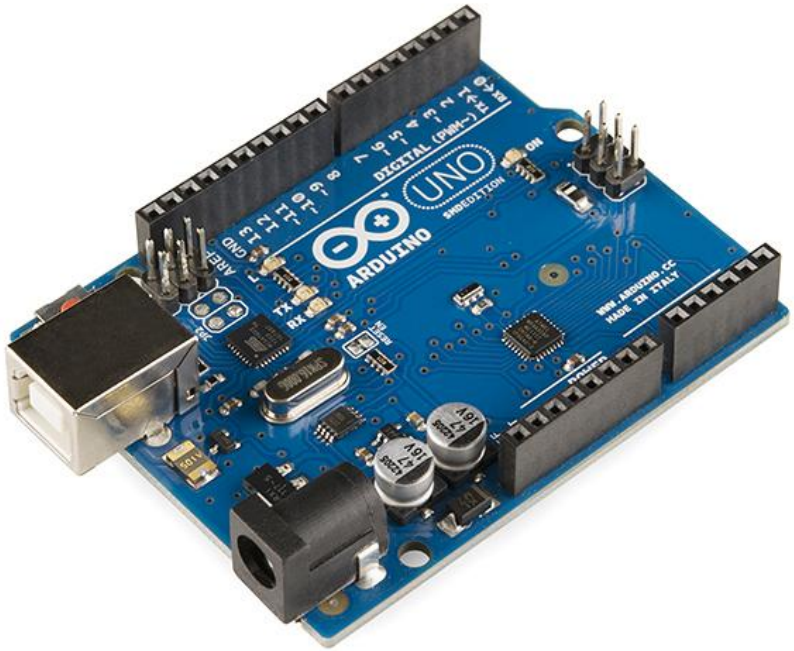
- Machine learning auto-maps unfamiliar data schemas between devices
- Protocol translation moves closer to devices for speed & privacy
- Standardized virtual models synchronize state across platforms
- Next-gen networks natively support diverse IoT traffic classes

Summary

1. Interoperability spans four levels: technical, syntactic, semantic, and organizational.
2. MQTT, CoAP, AMQP and HTTP solve messaging; Zigbee, Thread, BLE and LoRaWAN solve connectivity.
3. Bodies like IEEE, IETF, W3C, oneM2M and OCF/CSA define the rules devices agree to follow.
4. Gateways, ontologies, and unified APIs bridge the gaps standards leave behind.
5. Matter shows convergence is possible when industry aligns around one standard.

Introduction to ARDUINO

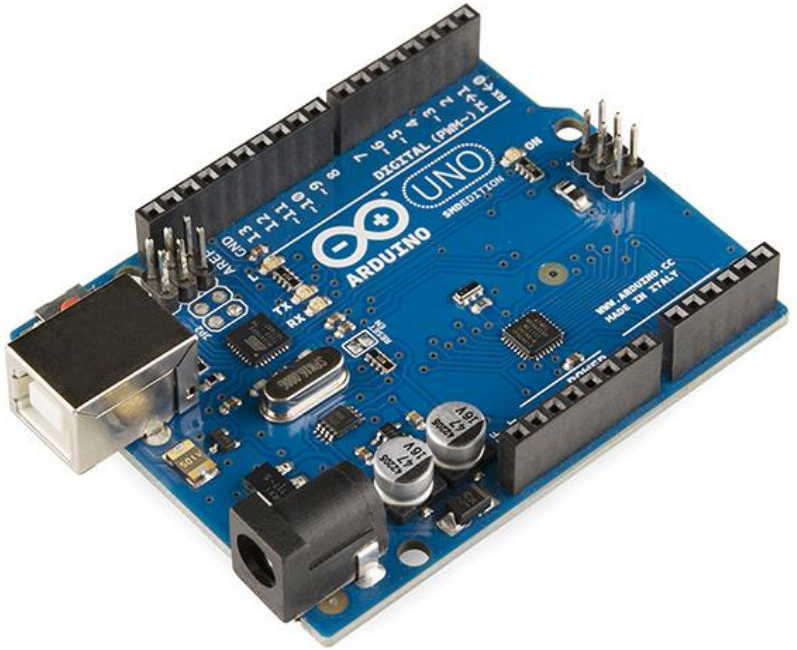
- Arduino UNO-Hardware



- Arduino IDE-Software



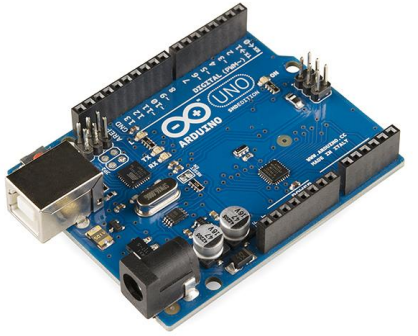
Arduino Board



- Arduino is an Italian open-source hardware and software company, project, and user community that designs and manufactures single-board microcontrollers and microcontroller kits for building digital devices

•Arduino was created in the year 2005 by two Italian engineers David Cuartielles and Massimo Banzi

Arduino Board



- Basically, Arduino is Microcontroller based board.
- Microcontroller is microprocessor with on chip memory, RAM and some other peripheral connected with it.
- The Arduino Uno is a microcontroller board based on the ATmega328 .
- The ATmega328 has Flash memory of 32 KB (with 0.5 KB used for the bootloader), Where program is stored.
- It also has 2 KB of SRAM- sketch creates and manipulates variables when it runs.
- 1 KB of EEPROM, memory whose values are kept when the board is turned off.

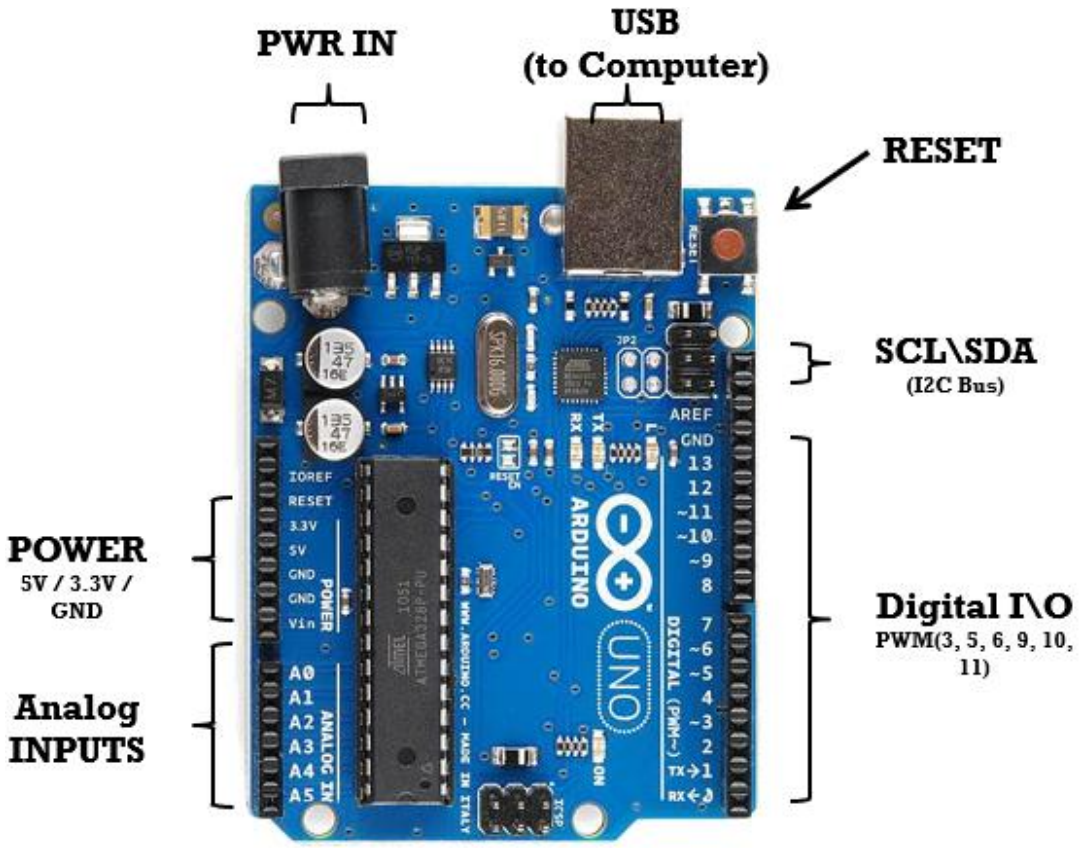
Arduino Board

- **Most Popular Arduino Boards**



Arduino UNO

- What does it have?
  - 14 Digital In/Out pins (6 can be used as PWM)
  - 6 Analog Inputs
  - A USB Connection
  - A Power Jack
  - Reset Button
  - On-board LED
  - SCL/SDA pins (Serial Clock/ Serial Data pins)
- It contains everything needed to support the microcontroller
- simply connect it to a computer with
  - a USB cable
  - a AC-to-DC adapter
  - battery



How to code in Arduino ?

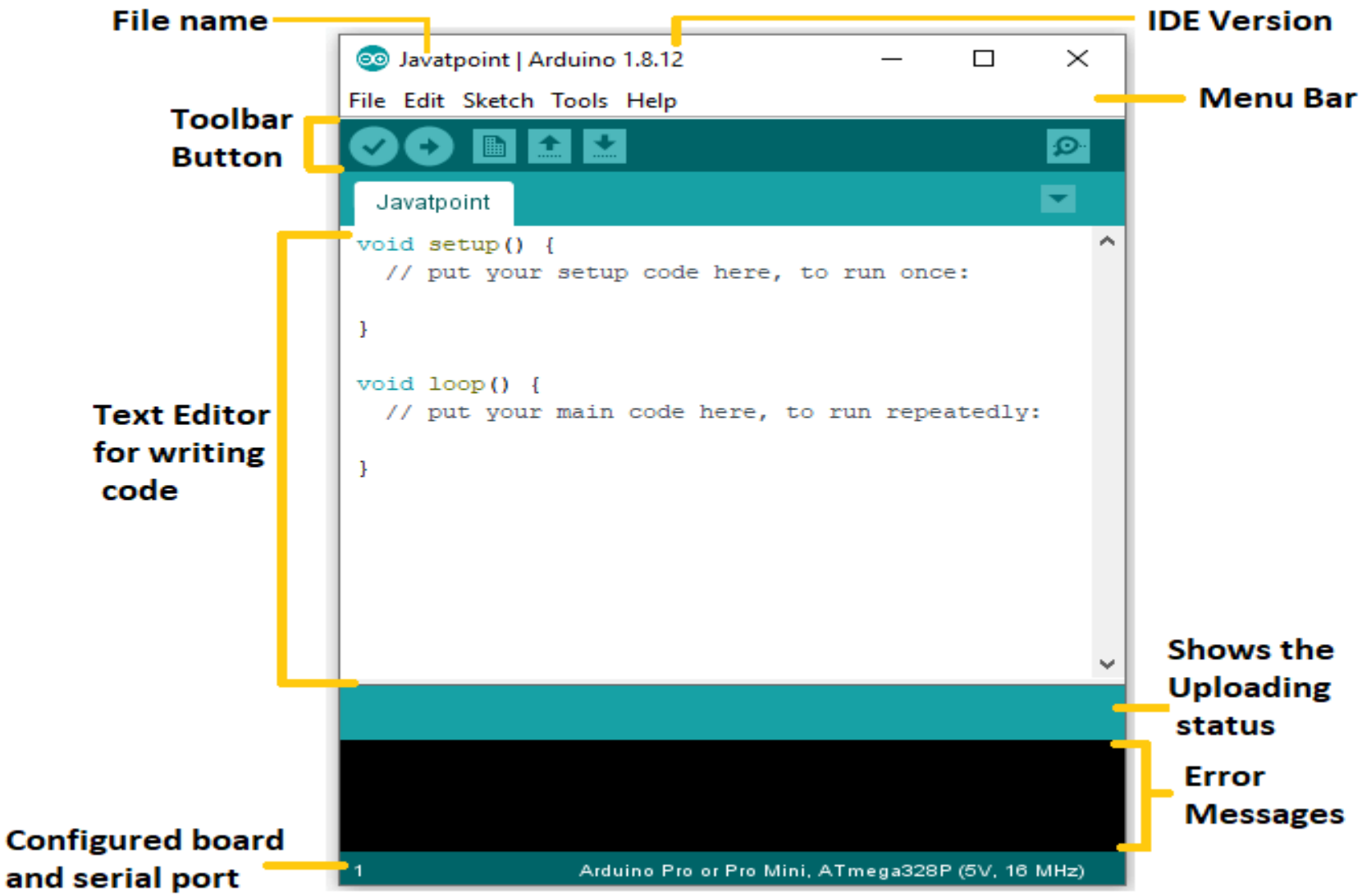
- Need to download Arduino IDE (Integrated Development Environment).
- Arduino IDE is available for all Mac, Windows and Linux.



What is IDE ?

- IDE contains
- Text editor: Where the user writes and edits source code
- Compiler/ interpreter: Translates the source code into a language that a computer can execute
- Debugger: Investigates logic errors.
- Linker: Tools for linking the library files.
- Locator: Map maker-Decides exactly where each bit of code will live in memory
- Loader: Mover-Physically "burns" or writes the code onto the Arduino chip
- Version control integration: Tools for integrating with version control systems

Arduino IDE


File name
IDE Version
Javatpoint | Arduino 1.8.12
File Edit Sketch Tools Help
Menu Bar
Toolbar Button
Javatpoint
void setup() {
// put your setup code here, to run once:
}
void loop() {
// put your main code here, to run repeatedly:
}
Text Editor for writing code
Shows the Uploading status
Error Messages
Configured board and serial port
1
Arduino Pro or Pro Mini, ATmega328P (5V, 16 MHz)

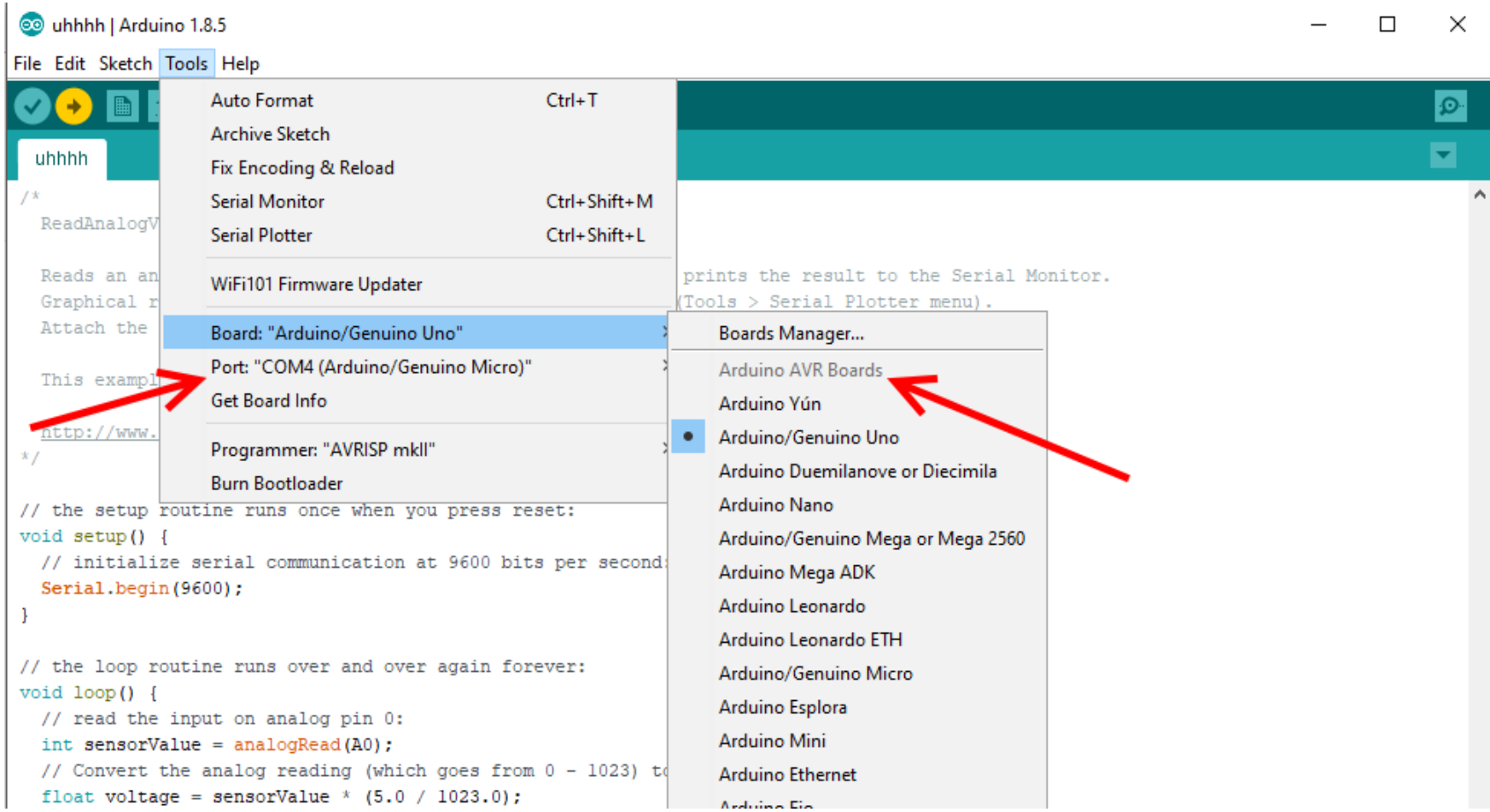
Arduino IDE

Part of IDE	Real-World Equivalent	What it does
Editor	A Notebook	Where you write your instructions.
Verify	A Grammar Teacher	Checks for mistakes before you finish.
Upload	A Delivery Truck	Sends the code to the Arduino hardware.
Serial Monitor	A Status Screen	Let's you see what the Arduino is "thinking."
Libraries	Expert Help	Adds new abilities for special sensors.



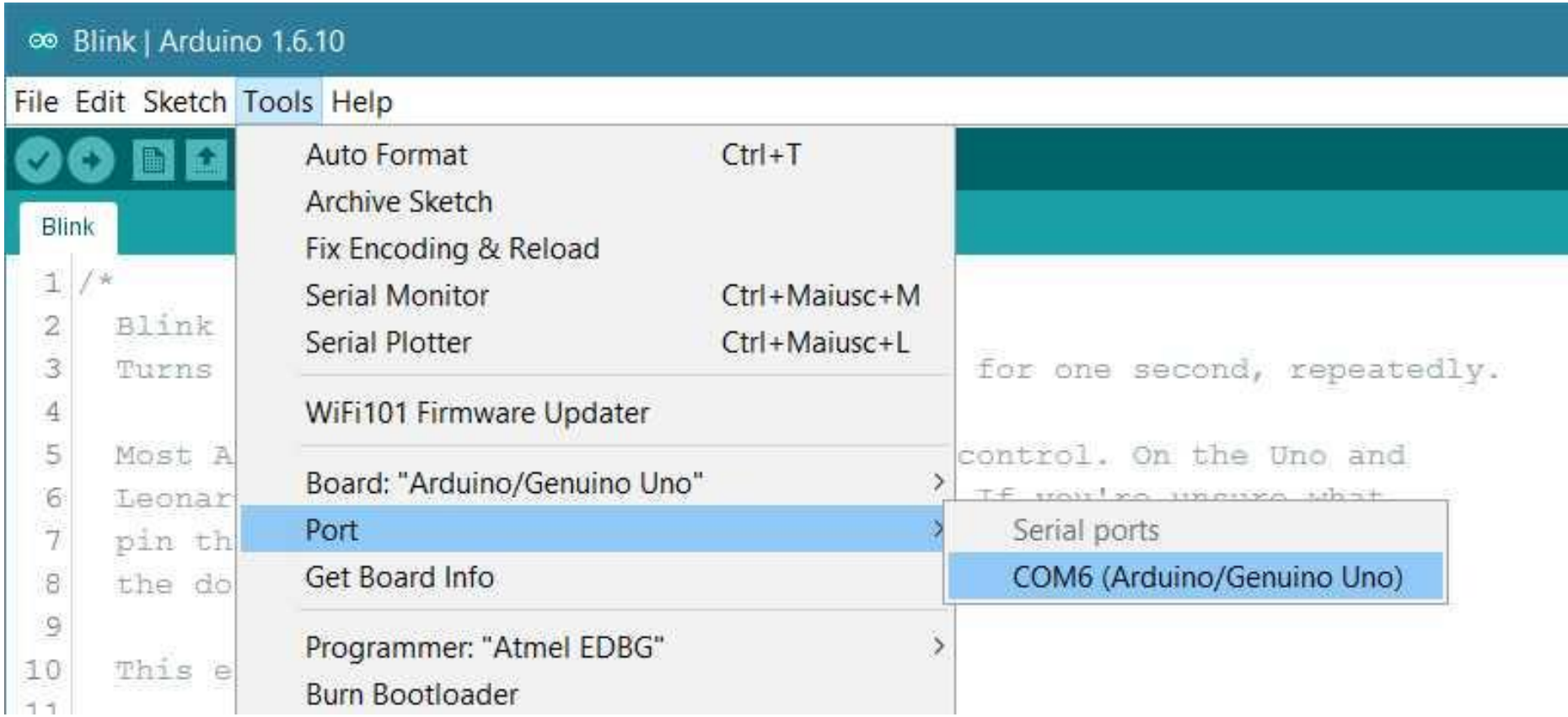
Arduino IDE

- **Selection of Target Board**



Arduino IDE

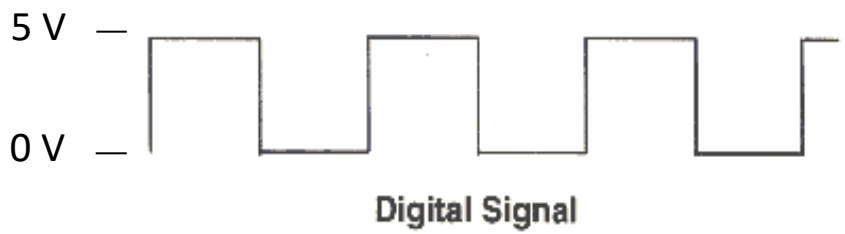
- **Selection of Communication Port**



Analog Vs Digital

- Microcontrollers are **digital** devices – ON or OFF. Also called – discrete.

- **Analog** signals are anything that can be a full range of values.



Digital Signal



Analog Signal

Analog Vs Digital

- Analog Sensors

Sensors	Variables
Mic	soundVolume
Photoresistor	lightLevel
Potentiometer	dialPosition
Temp Sensor	temperature
Flex Sensor	bend
Accelerometer	tilt/acceleration

- Digital Sensors

  - Digital sensors are more straight forward than Analog.
  - No matter what the sensor there are only two settings: On and Off

- Example, Push button, Switch

Serial communication

- "Serial" because data is broken into bits, each sent one after another in a single wire.
- Compiling turns your program into binary data (ones and zeros)
- Uploading sends the bits through USB cable to the Arduino
- The two LEDs near the USB connector blink when data is transmitted.
  - RX blinks when the Arduino is receiving data.
  - TX blinks when the Arduino is transmitting data

Programming with Arduino

- A variant of the C++ programming language, along with special methods and functions.
- Arduino code is sometimes referred to as "embedded C" or "embedded C++" because it shares similar syntax and structure with C/C++
- Sketch: The name for code files in Arduino

Structure of Arduino Program

- **void setup()**
- It runs only once when you power on the Arduino.
- You use it to initialize pins (e.g., "Pin 13 is an Output").

- **void loop()**
- It runs over and over again forever.
- As soon as it reaches the bottom, it jumps back to the top.

Arduino Programming

Variables (The Memory Buckets)

- **`int` (Integer):** Stores whole numbers
  - Ex: temperature: 25
- **`float`:** Stores decimals
  - Ex: precise voltage: 3.75
- **`bool` (Boolean):** Stores only two values
  - Ex: `true` or `false` (like "Is the light on?")

Arduino Programming

Digital vs. Analog (The Language)

- **Digital:** Only two states—**HIGH** or **LOW**
  - Ex: Turning a light bulb ON or OFF -> digitalWrite(13, HIGH);
  - Reading a value from a digital Sensor -> button=digitalRead(A2);
- **Analog:** A range of values.
  - *Ex:* Checking exactly how bright the sun is.
  - sensor_value=analogRead(A0);
  - analogWrite(3, a); //PWM pin must be used

Arduino Programming

Control Logic (The Decision Maker)

This is the "Brain" part of the analogy. It uses **If-Statements**.

C++

```
if (lightLevel < 500)
{
digitalWrite(13, HIGH); // Turn on LED if it's dark
}
 else
{
 digitalWrite(13, LOW); // Turn off LED if it's bright
}
```

Arduino Programming

The Serial Monitor (The Diary)

- Since the Arduino doesn't have a screen, it uses the **Serial Monitor** to talk to your computer.
- You use the command `Serial.println("Hello!");` to see what is happening inside the chip.
- It is the best tool for **Debugging** logic errors.

Blinking of an LED

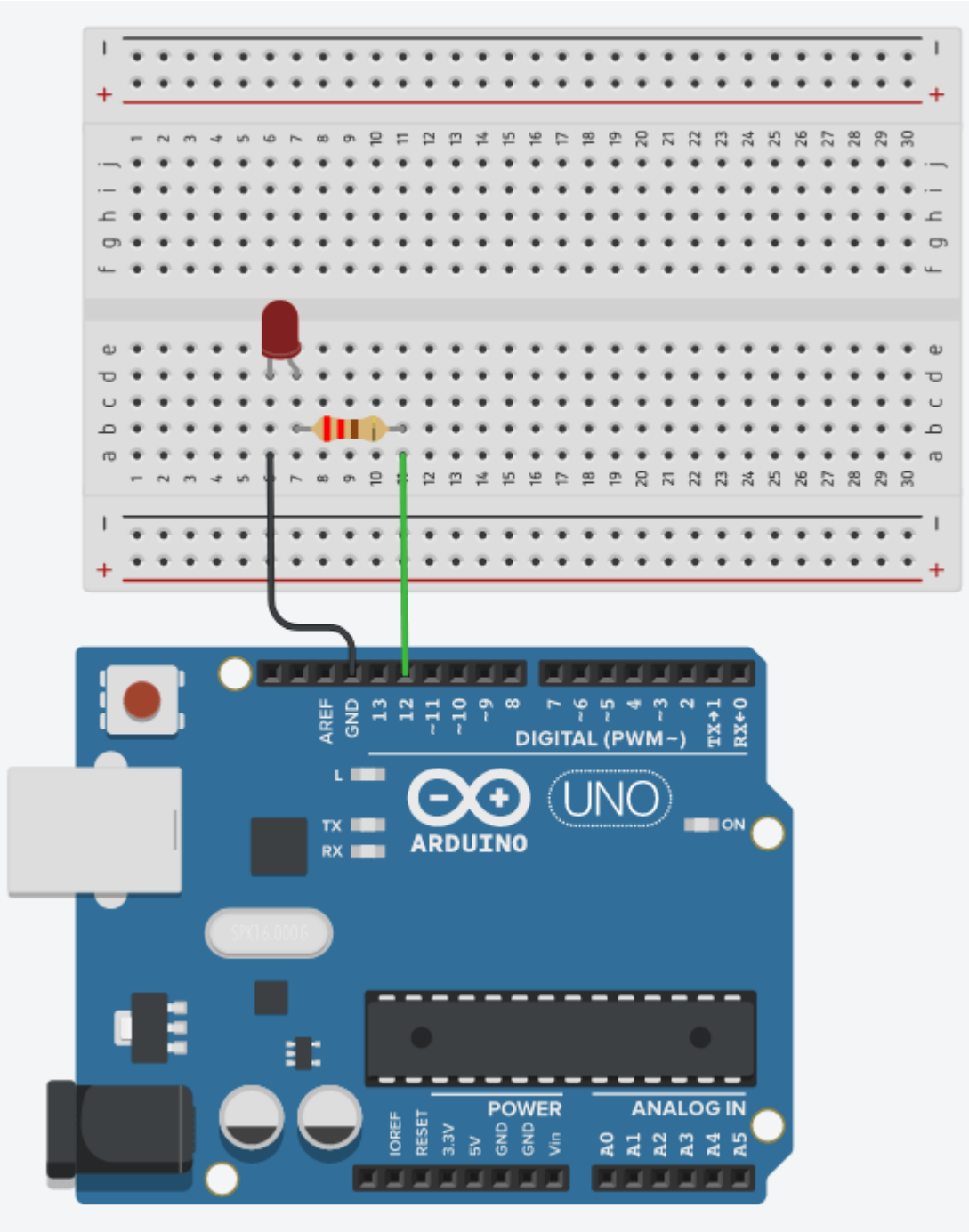


- LED blinking refers to the process of continuously turning an LED (Light Emitting Diode) ON and OFF in a repetitive pattern

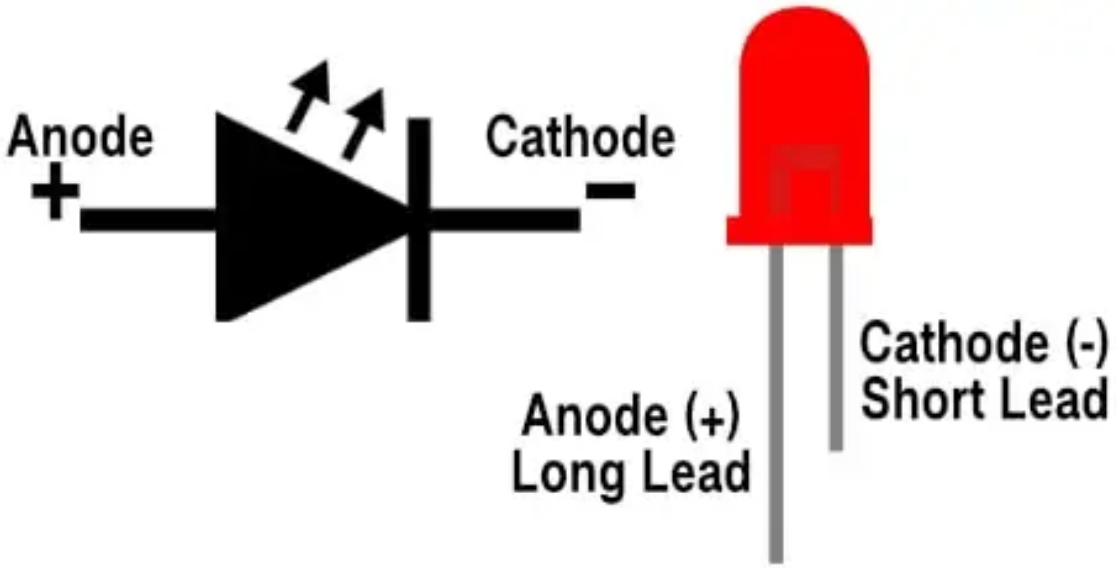
Blinking of an LED

- How to Connect LED to Arduino

- Anode of LED- To any digital pin of Arduino through Resistor
- Cathode of LED- To GND



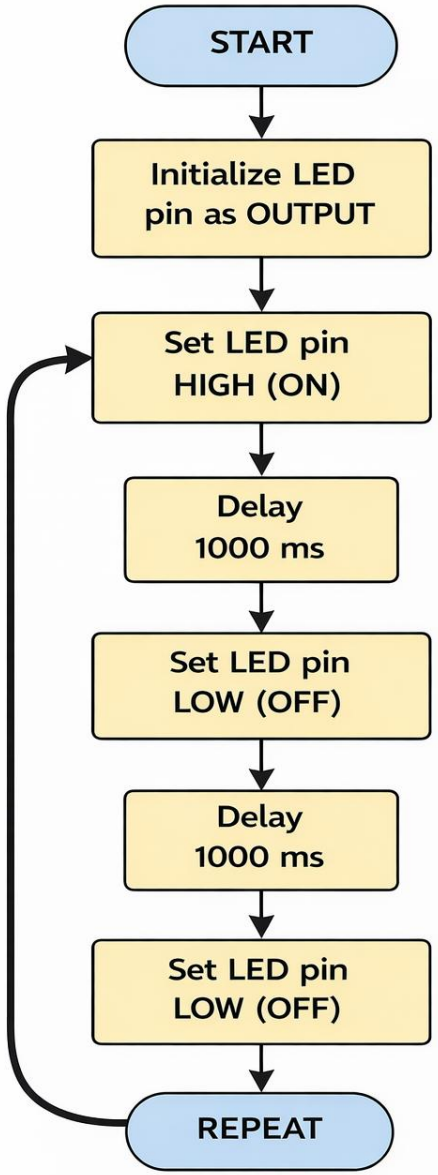
LED- Light Emitting Diode



- LED must always be connected in Forward bias
- Resistor is connected in series with LED, to limit the current flow, and to prevent the LED from burning out.

Algorithm & Flow chart-Blinking of an LED

1. **Start**
2. **Initialize** the digital pin (e.g., pin 13) as **OUTPUT**
3. **Set the LED pin HIGH** (turn ON the LED)
4. **Wait** for a fixed time (e.g., 1000 ms)
5. **Set the LED pin LOW** (turn OFF the LED)
6. **Wait** for the same fixed time (e.g., 1000 ms)
7. **Repeat steps 3 to 6 continuously**
8. **Stop**



Arduino code-Blinking of an LED

```
int LEDpin = 12;

int delayT = 1000;

void setup() {

pinMode(LEDpin, OUTPUT);

}

void loop() {

digitalWrite(LEDpin, HIGH);

delay(delayT);

digitalWrite(LEDpin, LOW);

delay(delayT);

}
```

Program elements of Arduino C++

Operators

- Arithmetic Operators: =, +, -, *, /, %
- Comparison Operator: ==, !=, <, >, <=, >=
- Boolean Operator: &&, ||, !
- Bitwise Operator: &, |, ^, ~, <<, >>,
- Compound Operator: ++, --, +=, -=, *=, /=, %=, |=, &=

Program elements of Arduino C++

Control Statement

- If statement
  - if(condition){
    Statements if the condition is true ;
    }
- If…Else statement
  - if(condition){
    Statements if the condition is true;
    }
    else{
    Statements if the condition is false;
    }

- If…….Elseif…..Else
  - if (condition1){
    - Statements if the condition1 is true;
    - }
    - else if (condition2){ Statements if the condition1 is false
    - and condition2 is true;
    - }
    - else{
    - Statements if both the conditions are false;
    - }

- Switch Case
  - Switch(choice)
    {
    case opt1:
    statement_1;break;
    case opt2:
    statement_2;break;
    case opt3:
    statement_3;break;
    .
    .
    .
    case default: statement_default;
    break;
    }

- Conditional Operator.
  - Val=(condition)?(Statement1):(Statement2)

Program elements of Arduino C++

Loops

- For loop
  - for(initialization; condition; increment){ Statement till the condition is true;
    }
- While loop
  - while(condition){
    Statement till the condition is true;
    }
- Do… While loop
  - do{
    Statement till the condition is true;
    }while(condition);
- Nested loop: Calling a loop inside another loop
- Infinite loop: Condition of the loop is always true, the loop will never terminate

Program elements of Arduino C++

Arrays

- Collection of elements having homogenous datatype that are stored in adjacent memory location.
- The conventional starting index is 0.
- Declaration of array:

  <Datatype> array_name[size];

  Ex: int arre[5];
- Alternative Declaration: int arre[]={0,1,2,3,4};
- int arre[5]={0,1,2};
- Multi-dimentional array Declaration:

  <Datatype> array_name[n1] [n2][n3]….;

  Ex: int arre[row][col][height];

Program elements of Arduino C++

String

- Array of characters with NULL as termination is termed as a String.
- Declaration using Array:
  - char str[]="ABCD";
  - char str[4];
    - str[0]='A';
    - str[1]='B';
    - str[2]='C';
    - str[3]=0;
- Declaration using String Object:
  - String str="ABC";
- Functions of String Object:
  - str.ToUpperCase(): change all the characters of str to upper case
  - str.replace(str1,str2): is str1 is the sub string of str then it will be replaced by str2
  - str.length(): returns the length of the string without considering null

Program elements of Arduino C++

Math Library

- To apply the math functions and mathematical constants, "**MATH.h**" header files is needed to be included.
- Functions:
  - cos(double radian);
  - sin(double radian);
  - tan(double radian);
  - fabs(double val);
  - fmod(double val1, double val2);
- Functions:
  - exp(double val);
  - log(double val);
  - log10(double val);
  - square(double val);
  - pow(double base, double power);

Program elements of Arduino C++

Random Number

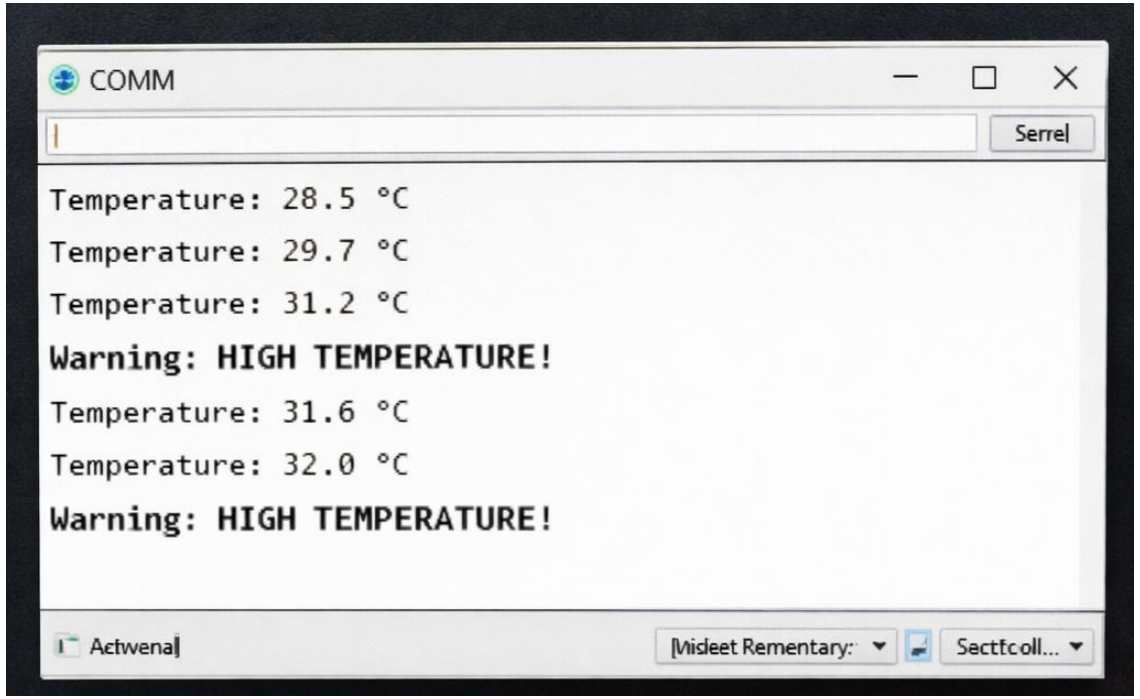
- randomSeed(int v): reset the pseudo-random number generator with seed value v
- random(maxi)=gives a random number within the range [0,maxi]
- random(mini,maxi)=gives a random number within the range [mini,maxi]

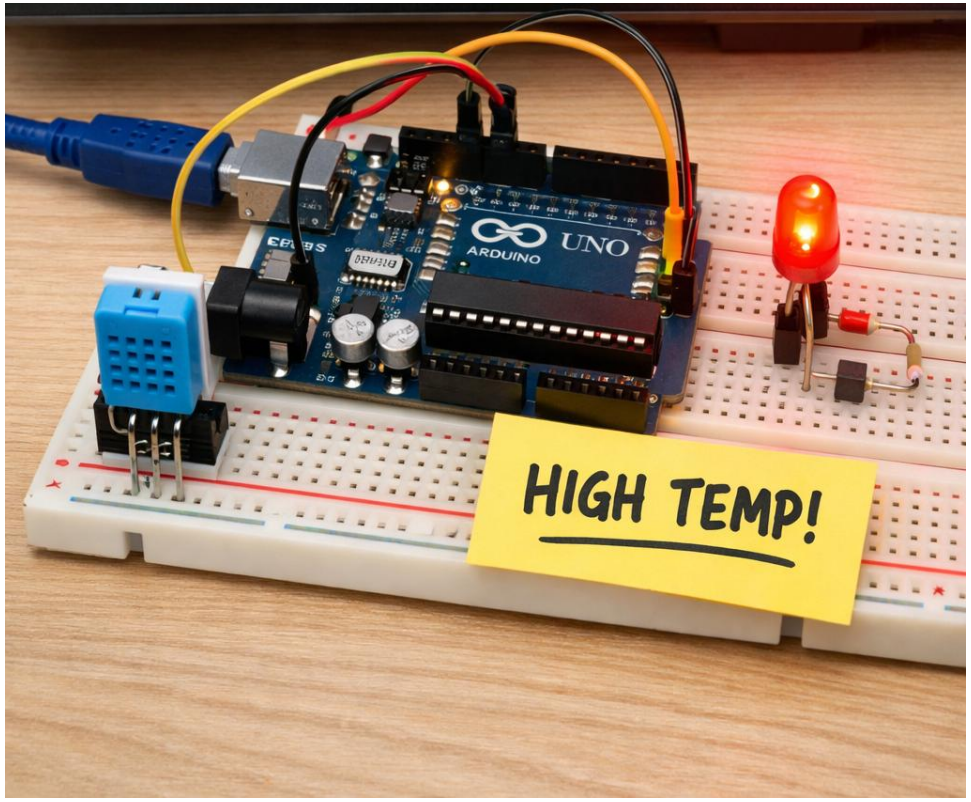
Program elements of Arduino C++

Interrupts

- An external signal for which system blocks the current running process to process that signal
- Types:
  - Hardware interrupt
  - Software interrupt
- digitalPinToInterrupt(pin): Change actual digital pin to the specific interrupt number.
- attachInterrupt(digitalPinToInterrupt(pin), ISR, mode);
  - ISR: a interrupt service routine have to be defined

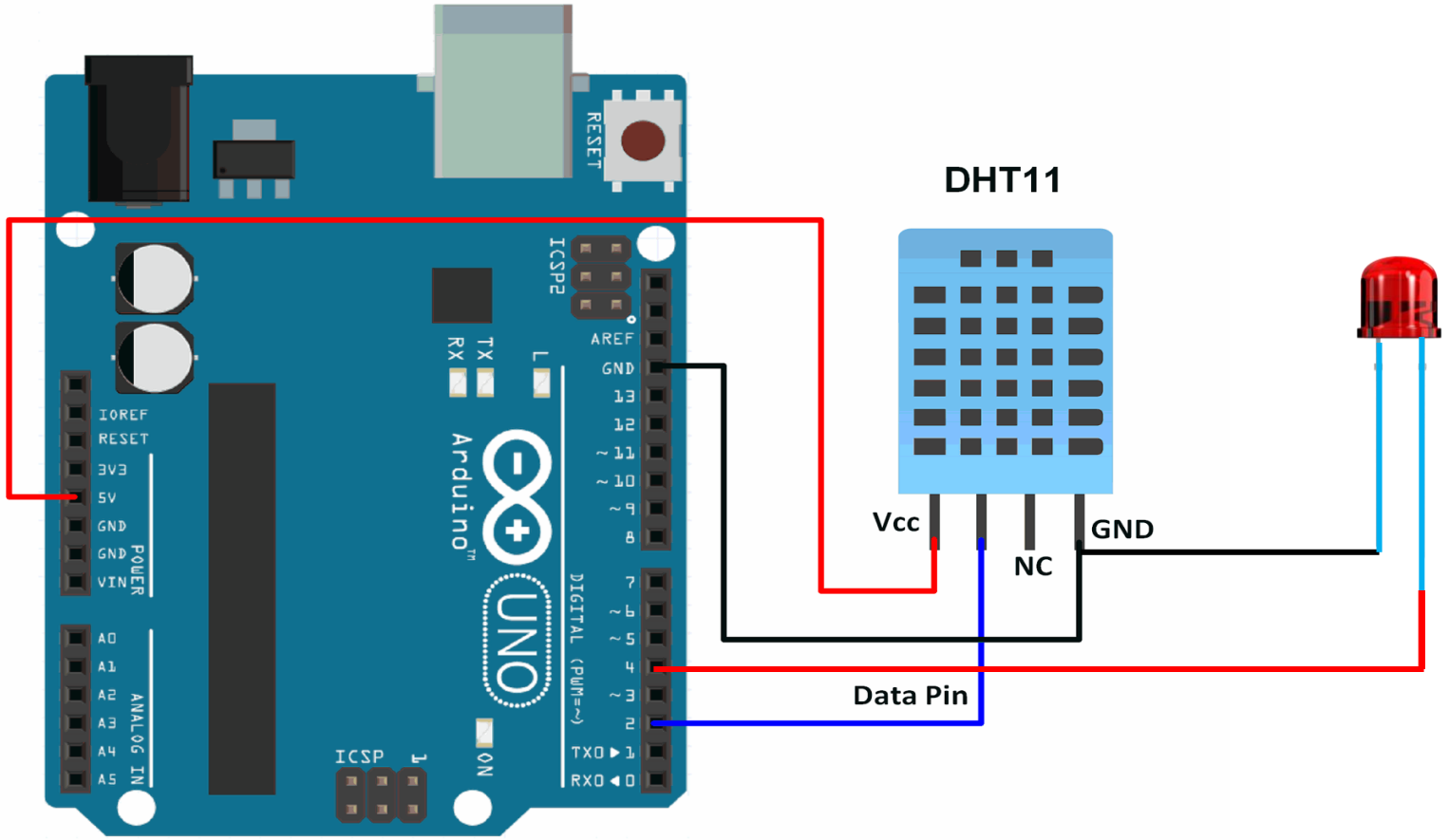
Measurement of Humidity and Room Temperature using DHT sensor and display the readings on Serial Monitor





Measurement of Humidity and Room Temperature using DHT sensor and display the readings on Serial Monitor

Schematic Diagram

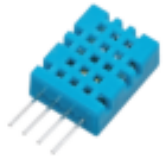


DHT Sensor

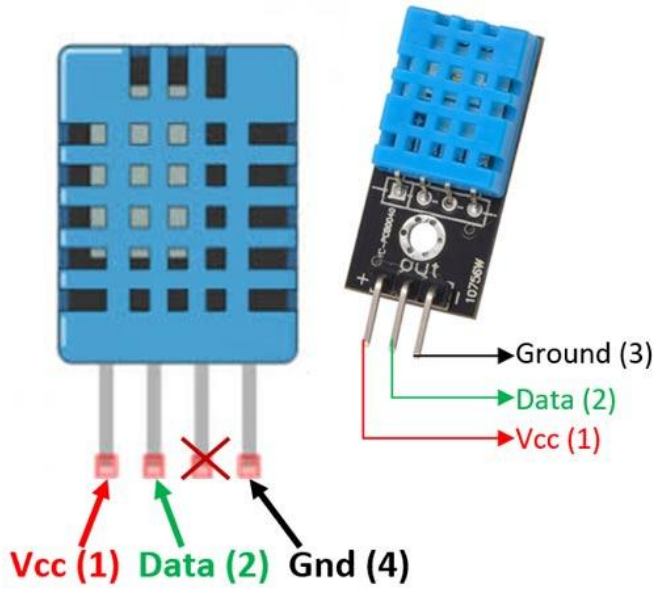
•Temperature and Humidity Sensor

•DHT11, DHT22

- Measures Temperature in the form of Resistance and
- Humidity in the form of Capacitance

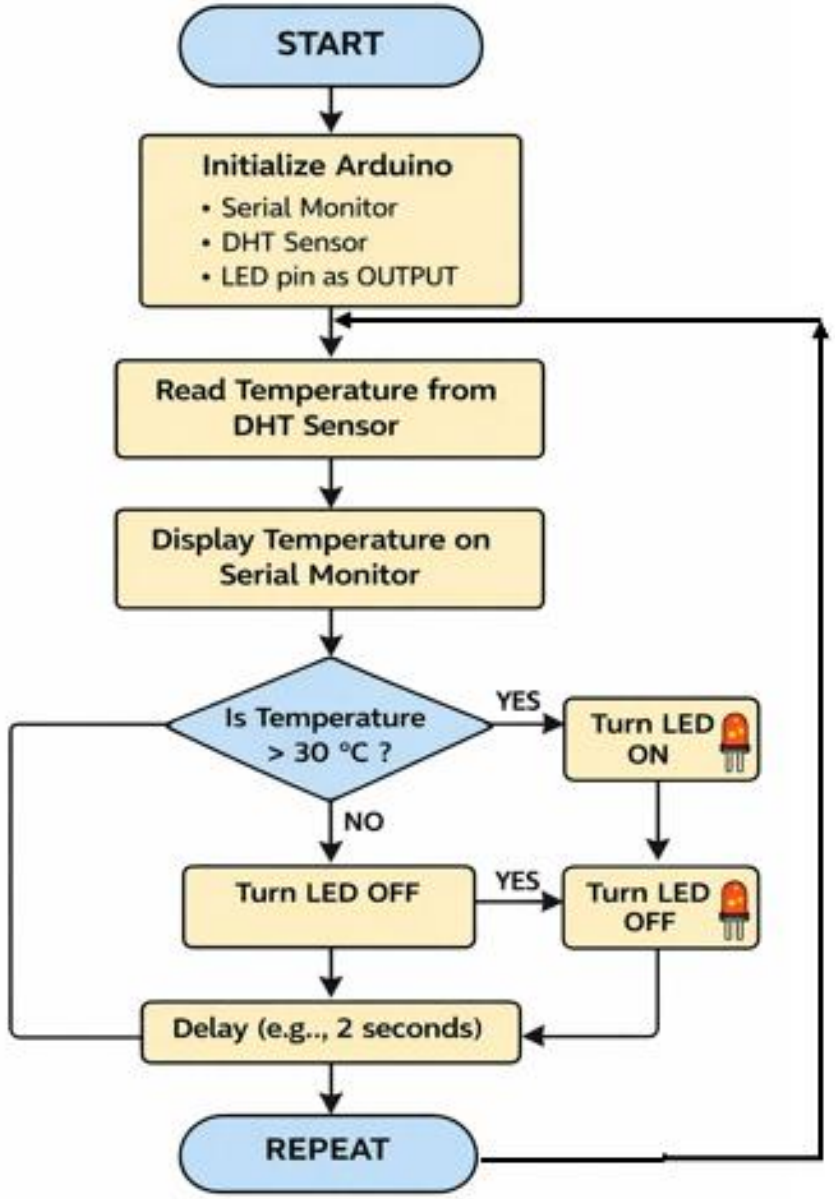
Image		
Part Number	DHT11	DHT22
Price	$1 to $5	$4 to $10
Sampling period	1 second	2 seconds
Current supply	0.5 ~ 2.5 mA	1 ~ 1.5 mA
Operating voltage	3 ~ 5.5 V	3 ~ 6 V
Temperature range	0 ~ 50 ºC（+/-2 ºC）	-40 ~ 80 °C（+/-0.5° C）
Humidity range	20 ~ 90%（+/-5%）	0 ~ 100%（+/-2%）
Body size	15.5mm x 12mm x 5.5mm	15.1mm x 25mm x 7.7mm
Resolution	Humidity: 1% Temperature: 1ºC	Humidity: 0.1% Temperature: 0.1° C

Pin out of DHT11



Measurement of Humidity and Room Temperature using DHT sensor and display the readings on Serial Monitor

Flow chart



Measurement of Humidity and Room Temperature using DHT sensor and display the readings on Serial Monitor

Source Code

```
#define PIN 2
#define TYPE DHT11
#include<DHT.h>
DHT dht(PIN, TYPE);
int led = 4;
void setup()
{
  Serial.begin(9600);
  dht.begin();
  pinMode(led, OUTPUT);
  pinMode(PIN,INPUT);
}
```

```
void loop()
{
  float tc = dht.readTemperature(false);
  float tf = dht.readTemperature(true);
  int h = dht.readHumidity();
  if(tc > 50.00)
  {
    tc = 0;
  }
```

Measurement of Humidity and Room Temperature using DHT sensor and display the readings on Serial Monitor

Source Code

```
  if(h > 100)
  {
    h = 0;
  }
  if(tc >= 30.00)
  {
Serail.println(“Warning: High Temperature”);
digitalWrite(led, HIGH);
  }
```

```
else
  {
    digitalWrite(led, LOW);
  }
  Serial.println();
  Serial.print("TEMPERATURE:  ");
  Serial.print(tc);
  Serial.print(" 'C , ");
```

Measurement of Humidity and Room Temperature using DHT sensor and display the readings on Serial Monitor

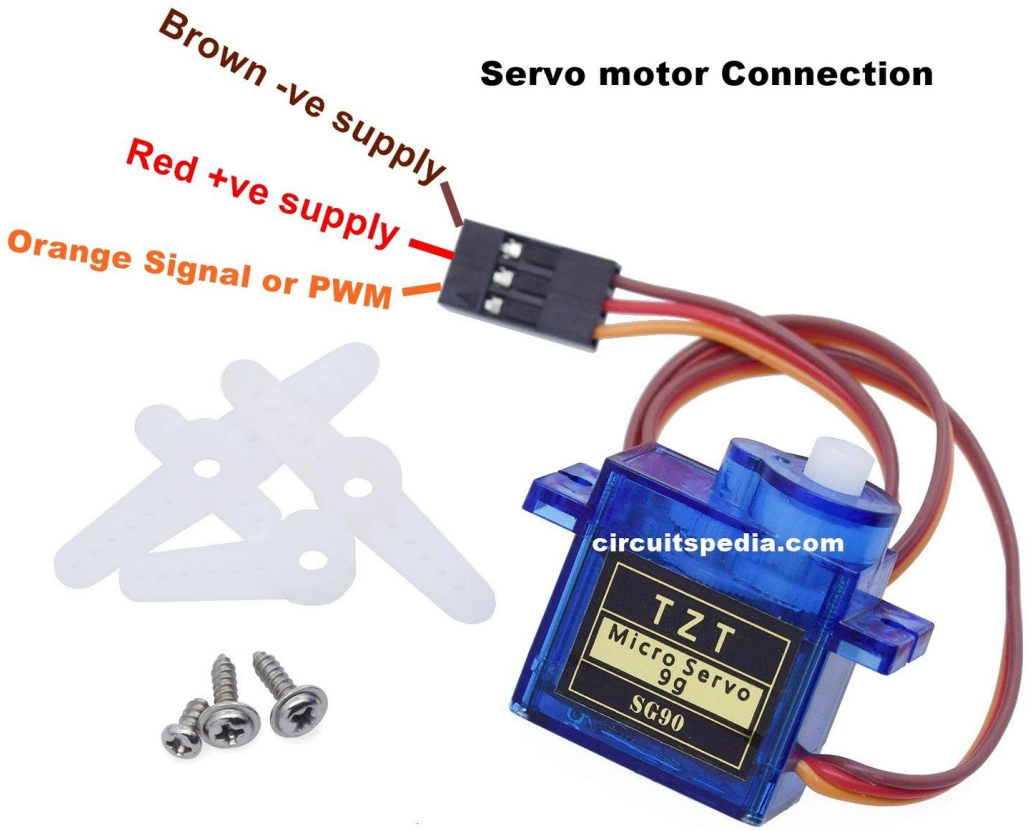
Source Code

```
  Serial.print(tf);
  Serial.print(" 'F");
  Serial.print(" and   ");
  Serial.print("HUMIDITY: ");
  Serial.print(h);
  Serial.print(" %");
  delay(2000);
}
```

Interfacing an Actuator(Servo Motor) with Arduino

Servo Motor

- High precision motor
- Provides rotary motion 0 to 180 degree
- 3 wires in the Servo motor
  - Black or the darkest one is Ground
  - Red is for power supply
  - Yellow for signal pin



Interfacing an Actuator(Servo Motor) with Arduino

Servo Library on Arduino

- Arduino provides different library- SERVO to operate the servo motor
- Create an instance of servo to use it in the sketch

Servo myservo;

Interfacing an Actuator(Servo Motor) with Arduino

Sketch: SERVO_ACTUATOR

```
#include <Servo.h>
//Including the servo library for the program
int servoPin = 12;

Servo ServoDemo; // Creating a servo object
void setup() {
// The servo pin must be attached to the servo before it can be used
ServoDemo.attach(servoPin);
}
```

```
void loop(){
  //Servo moves to 0 degrees
  ServoDemo.write(0);
  delay(1000);

  // Servo moves to 90 degrees
  ServoDemo.write(90);
  delay(1000);

  // Servo moves to 180 degrees
  ServoDemo.write(180);
  delay(1000);
```

Interfacing an Actuator(Servo Motor) with Arduino

Sketch: SERVO_ACTUATOR (contd..)

- Create an instance of Servo
- The instance must be attached to the pin before being used in the code
- Write() function takes the degree value and rotates the motor accordingly

File Edit Sketch Tools Help

SERVO_ACTUATOR

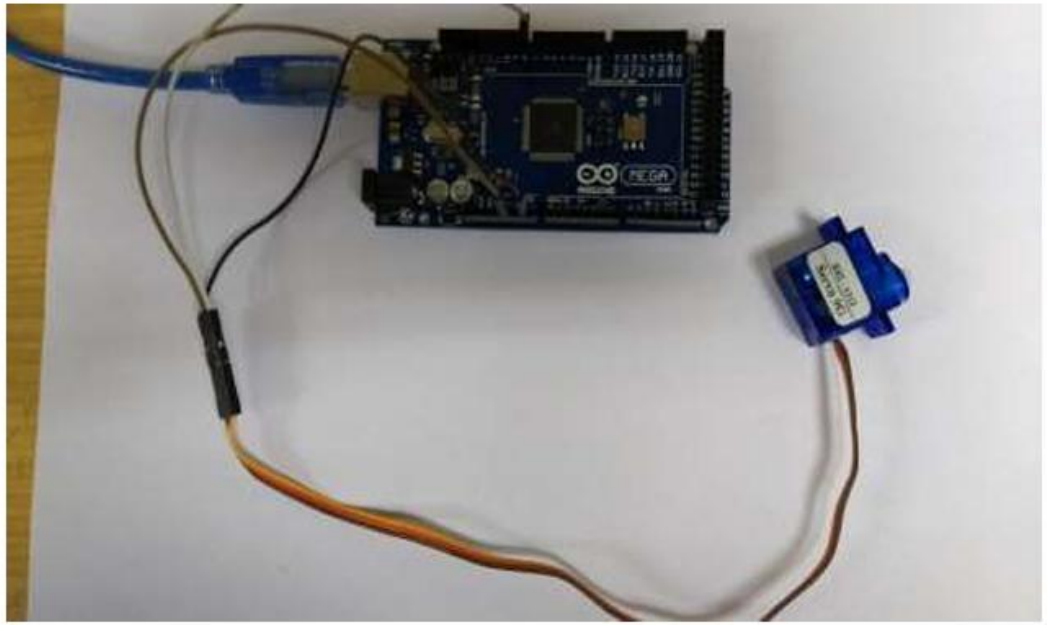
```
#include <Servo.h> //Including the servo library for the program
int servoPin = 8;

Servo ServoDemo; // Creating a servo object
void setup() {
   // The servo pin must be attached to the servo before it can be used
   ServoDemo.attach(servoPin);
}
void loop(){
   //Servo moves to 0 degrees
   ServoDemo.write(0);
   delay(1000);
   // Servo moves to 90 degrees
   ServoDemo.write(90);
   delay(1000);
   // Servo moves to 180 degrees
   ServoDemo.write(180);
   delay(1000);
}
```

Interfacing an Actuator(Servo Motor) with Arduino

Connection

- Connect the Ground of the servo to the ground of the Arduino board.
- Connect the power supply wire to the 5V pin of the board.
- Connect the signal wire to any digital output pin (we have used pin 8).



Interfacing an Actuator(Servo Motor) with Arduino

Board Setup

- Connect the board to the PC
- Set the port and board type
- Verify and upload the code

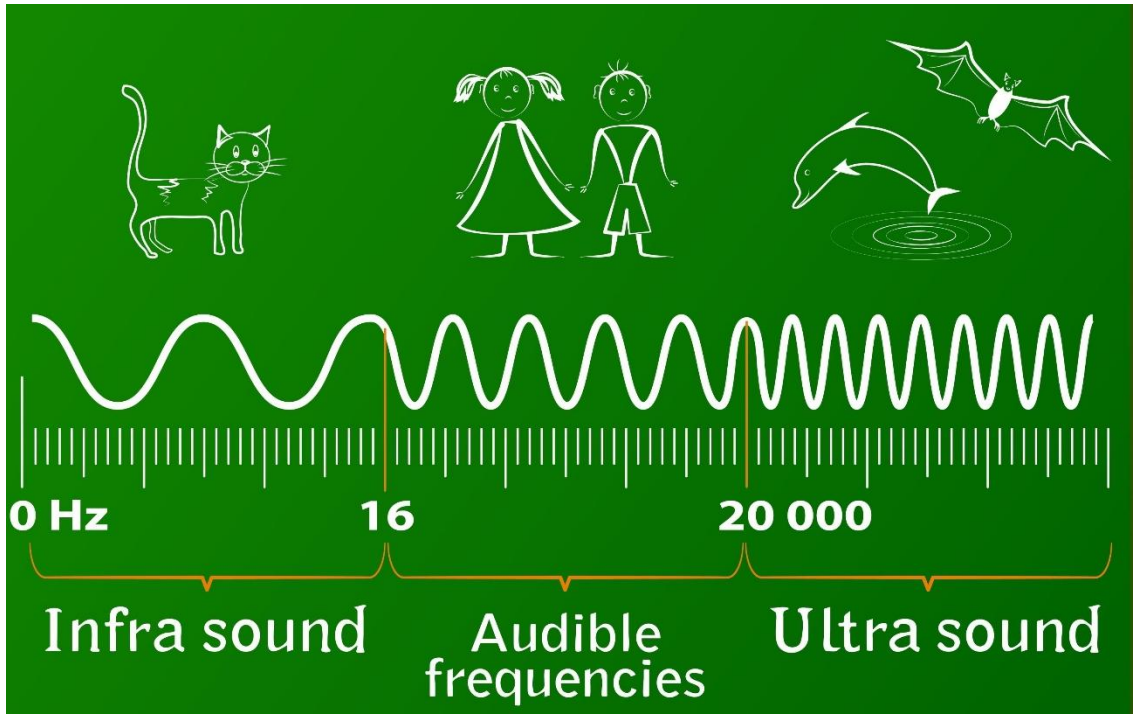
Interfacing an Actuator(Servo Motor) with Arduino

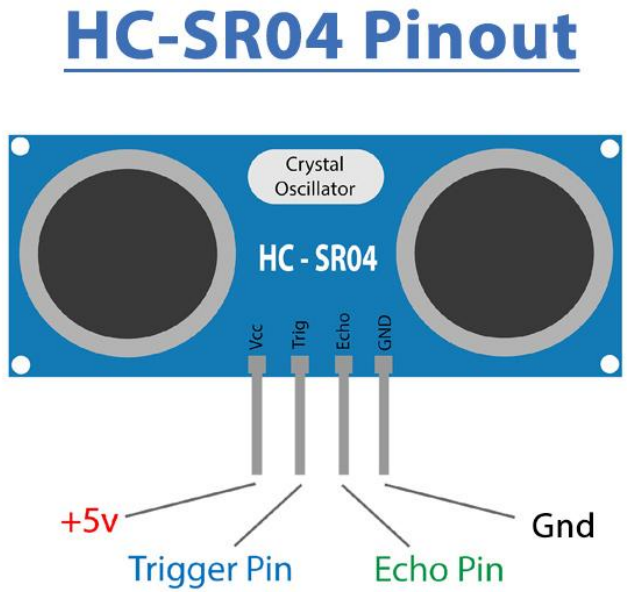
Output

The motor turns 0, 90 and 180 degrees
with a delay of 1 second each.



Measurement of Distance using Ultrasonic Sensor



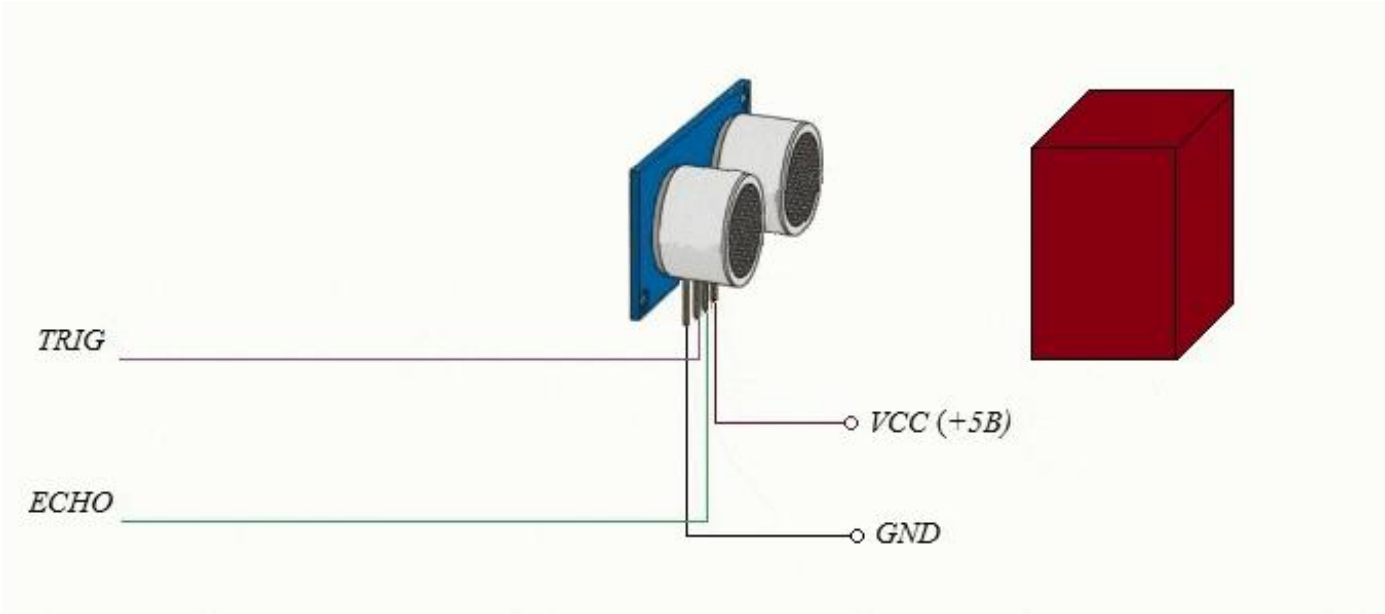




Ultrasonic Sensor-Working

Works on the Principle

Distance = Speed × Time



- The Ultrasonic transmitter transmits an ultrasonic wave,
- this wave travels in air and when it gets objected by any material it gets reflected back toward the sensor
- this reflected wave is observed by the Ultrasonic receiver module

Ultrasonic Sensor-Working

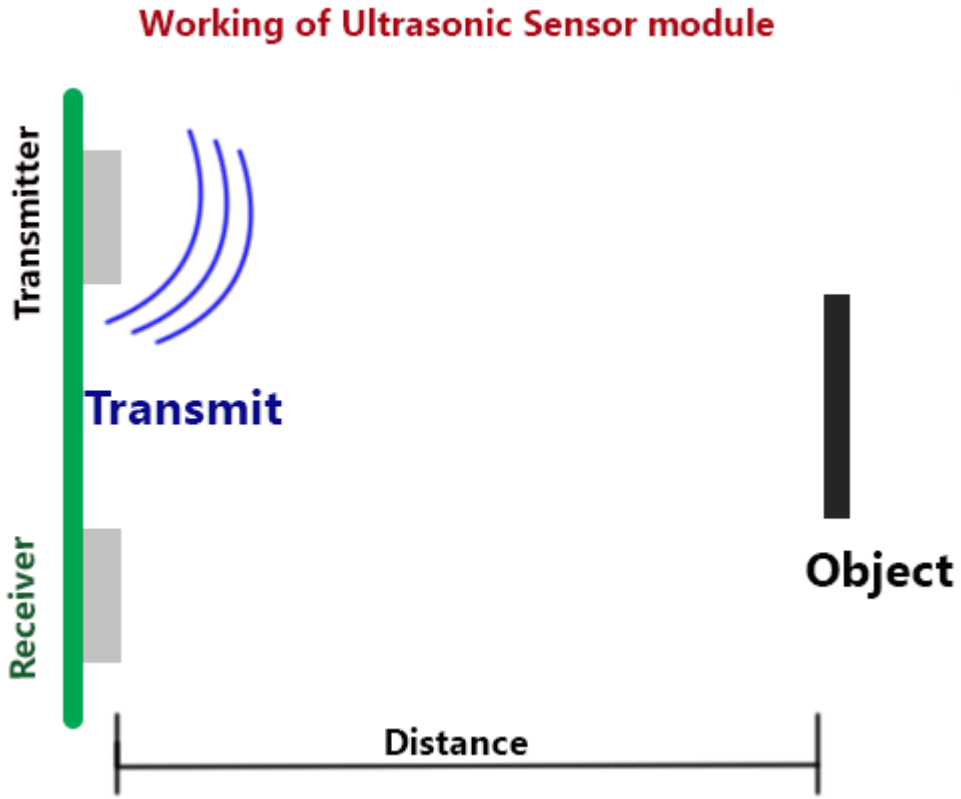
Ultrasonic sensors use sound waves that are above the range of human hearing

Speed of the ultrasonic sound wave S=344 m/s

Distance travelled by the ultrasonic sound wave=2D

Time taken = T

Distance D= (Time x Speed Of Sound) / 2



Ultrasonic Sensor-Specifications

HC-SR04 Sensor Features

Operating voltage: +5V

Theoretical Measuring Distance: 2cm to 450cm

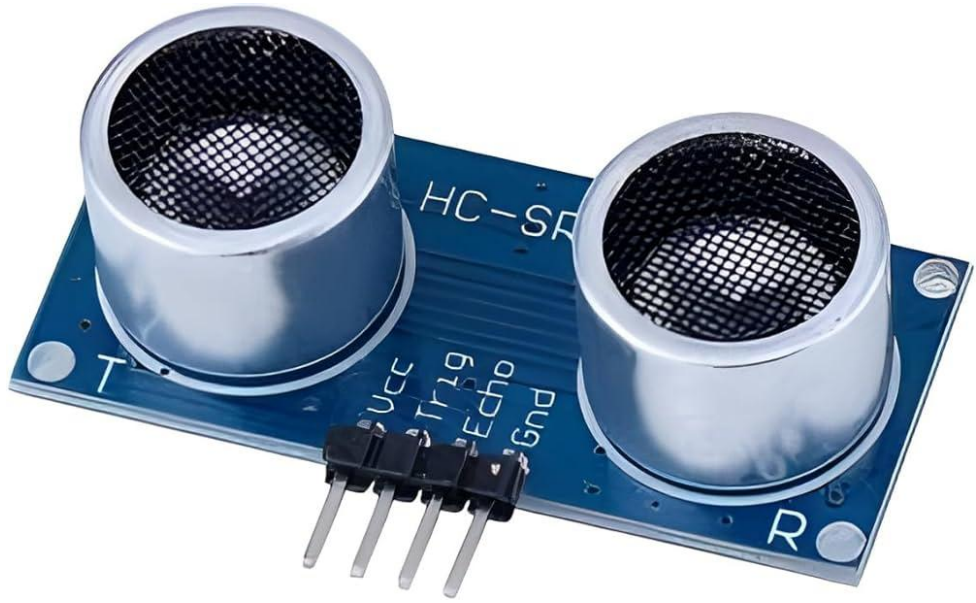
Practical Measuring Distance: 2cm to 80cm

Accuracy: 3mm

Measuring angle covered: <15˚

Operating Current: <15mA

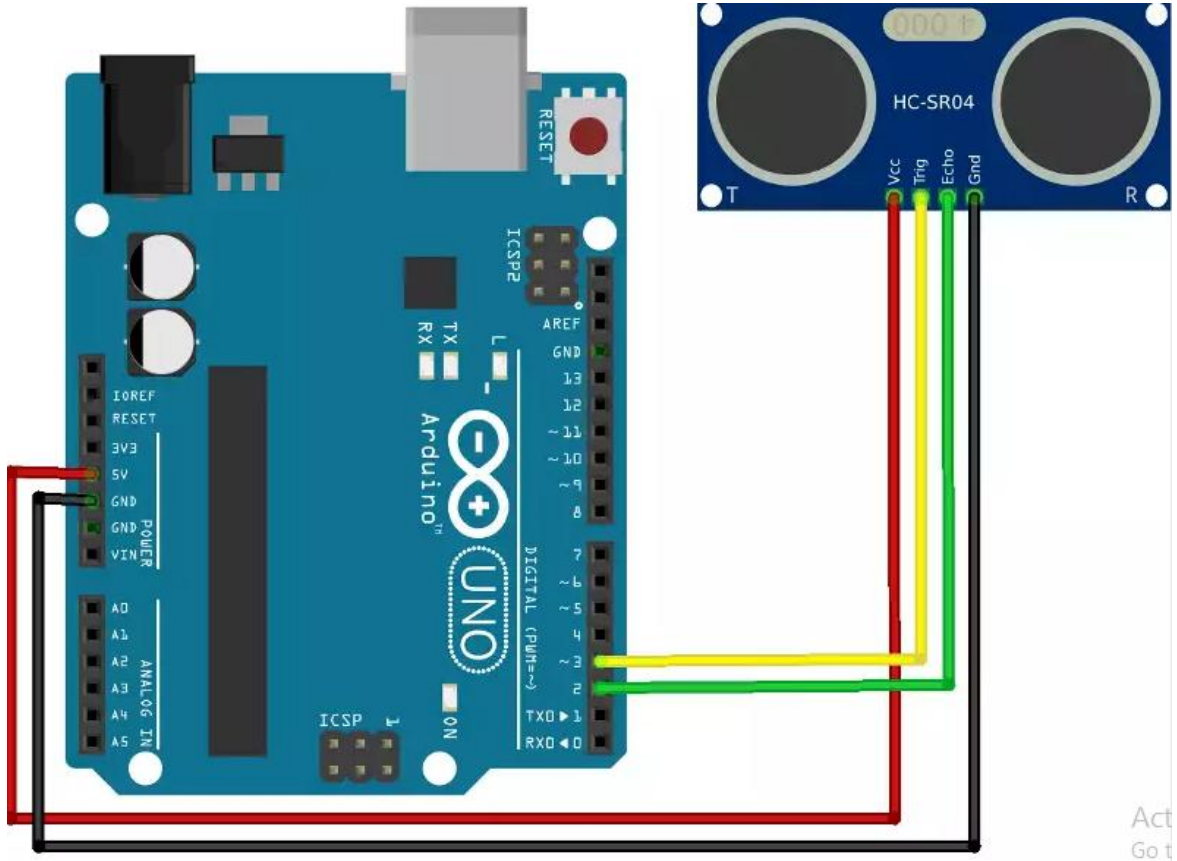
Operating Frequency: 40Hz



Measurement of Distance using Ultrasonic Sensor

- How to Connect HC-SR04 to Arduino

- Trig pin- any one digital pin of Arduino, 3
- Echo pin- any other digital pin of Arduino, 2
- Vcc-to 5V pin of Arduino
- GND- GND of Arduino



Arduino code-Measuring Distance using USS

```
#define echoPin    2 // attach pin D2 Arduino to pin Echo of HC-SR04
#define trigPin       3 // attach pin D3 Arduino to pin Trig of HC-SR04
  long duration; // Variable to store time taken to the pulse to reach receiver
 int distance; // Variable to store distance calculated using formula
 void setup()
{
    pinMode(trigPin, OUTPUT); // Sets the trigPin as an OUTPUT
    pinMode(echoPin, INPUT); // Sets the echoPin as an INPUT
    pinMode(led,OUTPUT);
       Serial.begin(9600);
        Serial.println("Distance measurement using Arduino Uno.");
    delay(500);
}
```

Arduino code-Measuring Distance using USS

```
 void loop()
{
   digitalWrite(trigPin, LOW);
   delayMicroseconds(2); // wait for 2 ms to avoid collision in serial monitor
    digitalWrite(trigPin,HIGH); // turn on the Trigger to generate pulse
   delayMicroseconds(10); // keep the trigger "ON" for 10 ms to generate pulse for 10 ms.
    digitalWrite(trigPin,LOW); // Turn off the pulse trigger to stop pulse generation
    // If pulse reached the receiver echoPin become high Then pulseIn() returns the
   // time taken by the pulse to reach the receiver
```

Arduino code-Measuring Distance using USS

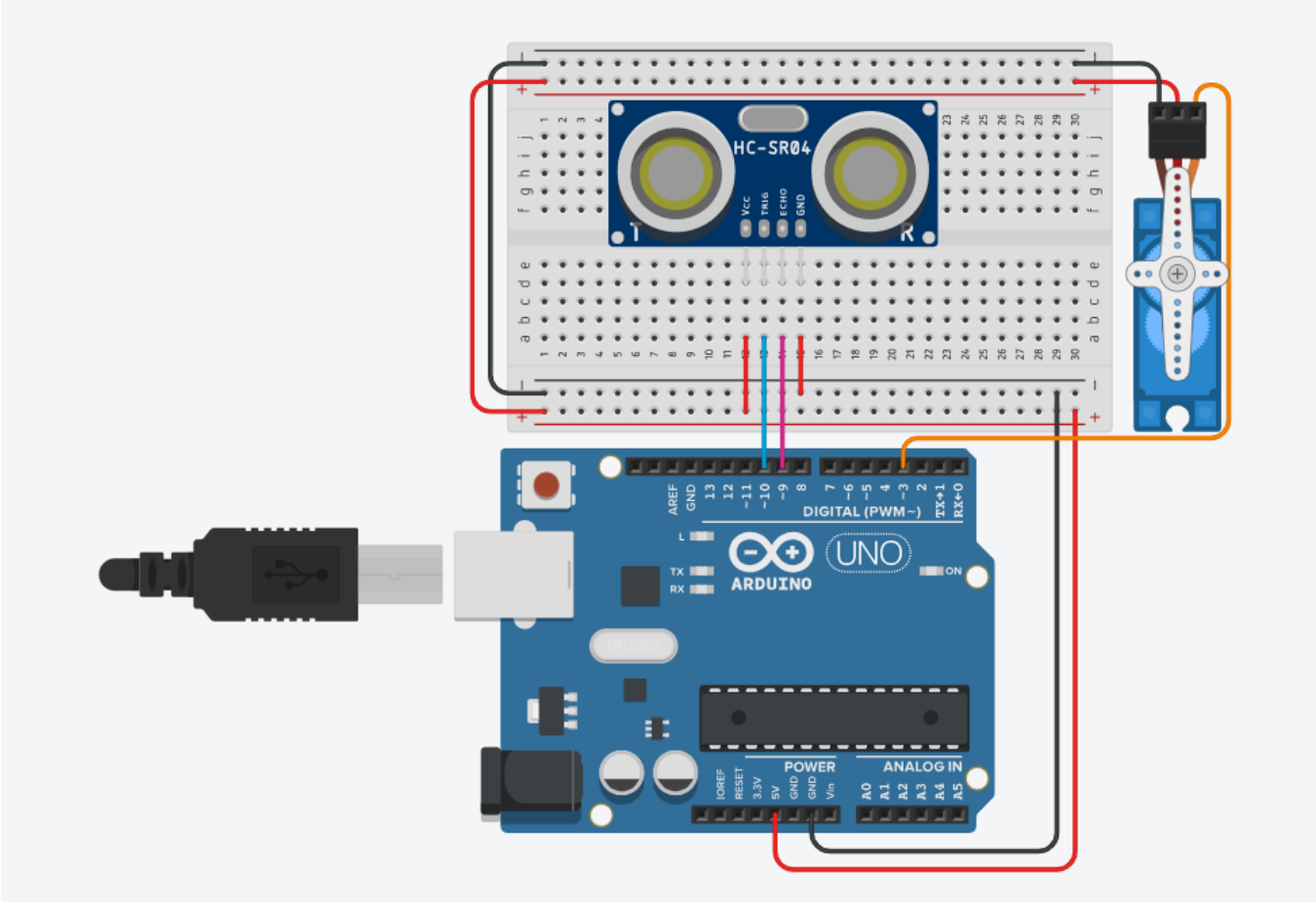
```
 duration = pulseIn(echoPin, HIGH);
 distance= duration * 0.0344 / 2; // Expression to calculate distance using time
  Serial.print("Distance: ");
 Serial.print(distance); // Print the output in serial monitor
 Serial.println(" cm");


delay(100);
}
```

Sensor triggering Actuator-Automatic Toll Gate

Toll Gate Design Specifications:

- If the distance < 30cm, the angle of the servo motor is 90.
- If the distance > 30, the angle of the servo motor is 0.



Sensor triggering Actuator-Automatic Toll Gate

```
#include <Servo.h>
int TRIG  = 10;
int ECHO  = 9;
int SERVO_orange = 3;
Servo myservo;
long duration = 0;
long distance =  0;
void setup()
 {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  myservo.attach(SERVO_orange);
  myservo.write(0);
  Serial.begin (9600);
}
```

Sensor triggering Actuator-Automatic Toll Gate

```
void loop() {

  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  duration = pulseIn(ECHO, HIGH);
  distance = (duration*0.0344/2);

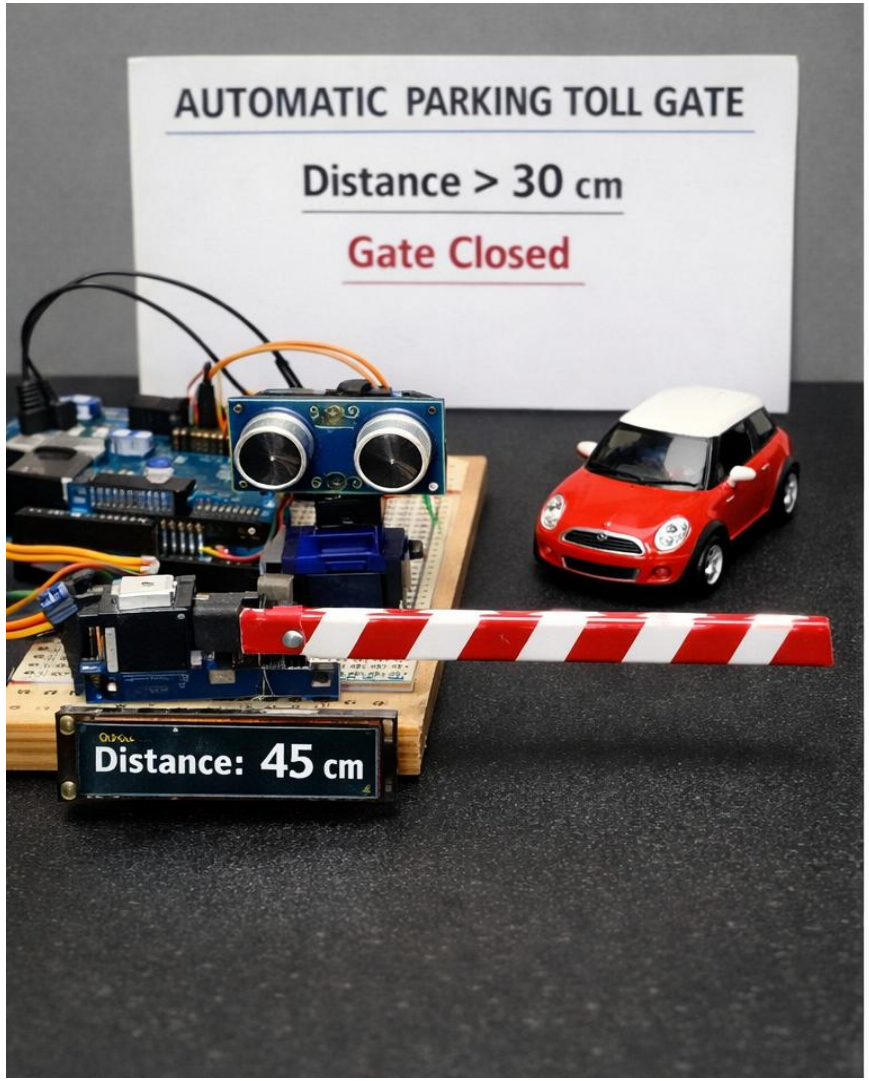
  if(distance< 30){
    myservo.write(90);
    Serial.println(“ TollGate is opened”);
  }
  else {
    myservo.write(0);
    Serial.println(“TollGate is closed”);
  }
```

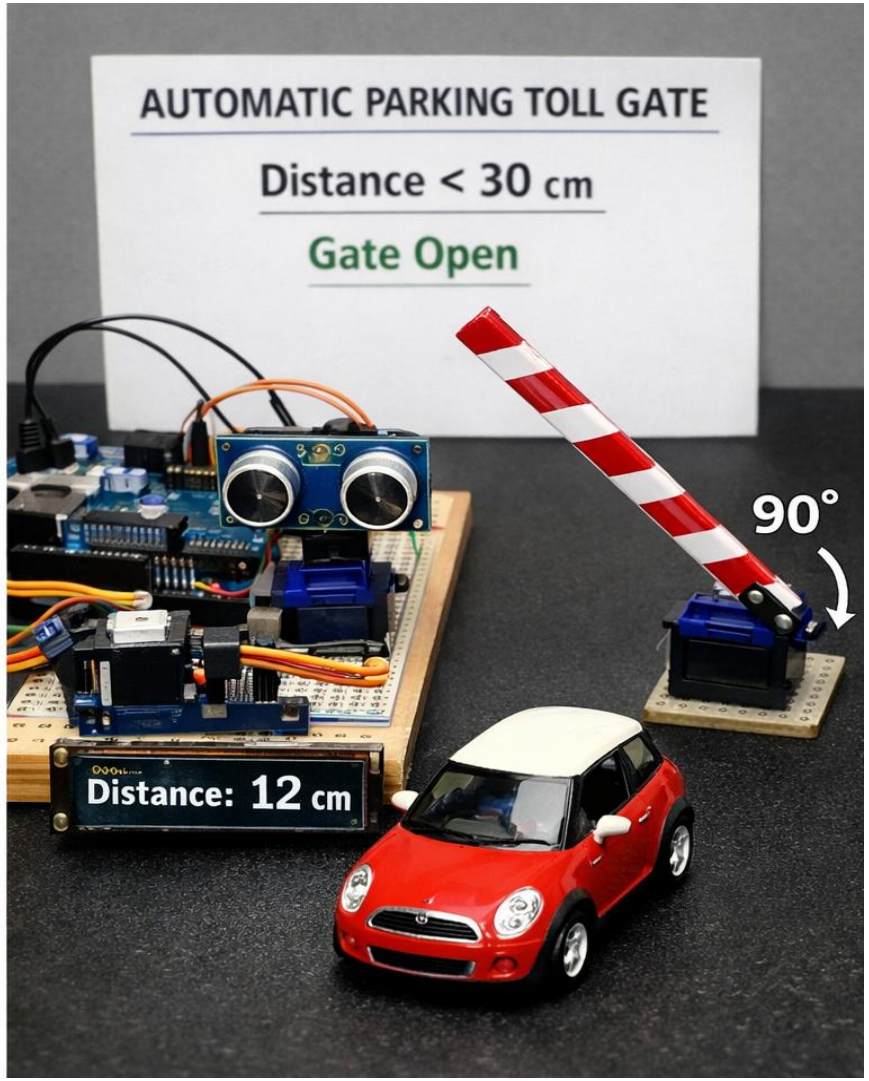
```
  Serial.print("distance: ");
   Serial.print(distance);
   Serial.println(" cm");

   delay(5000);
}
```

Sensor triggering Actuator-Automatic Toll Gate

Expected Output



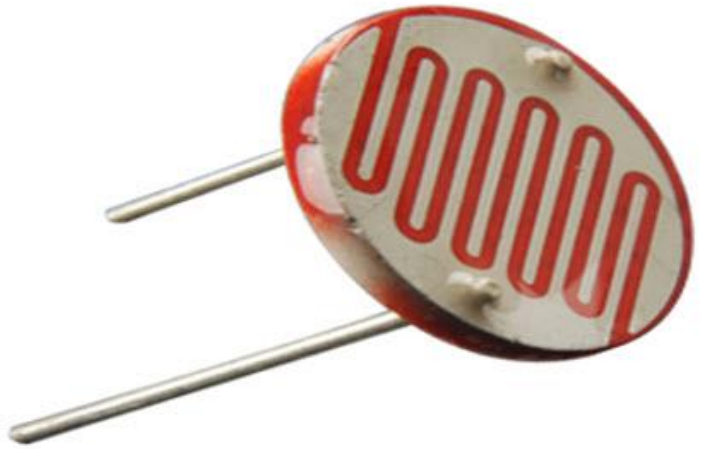


Interfacing an Analog Sensor with Arduino UNO

- LDR (Light Dependent Resistor) as the name states is a special type of resistor that works on the **photoconductivity** principle means that resistance changes according to the i**ntensity** of light.

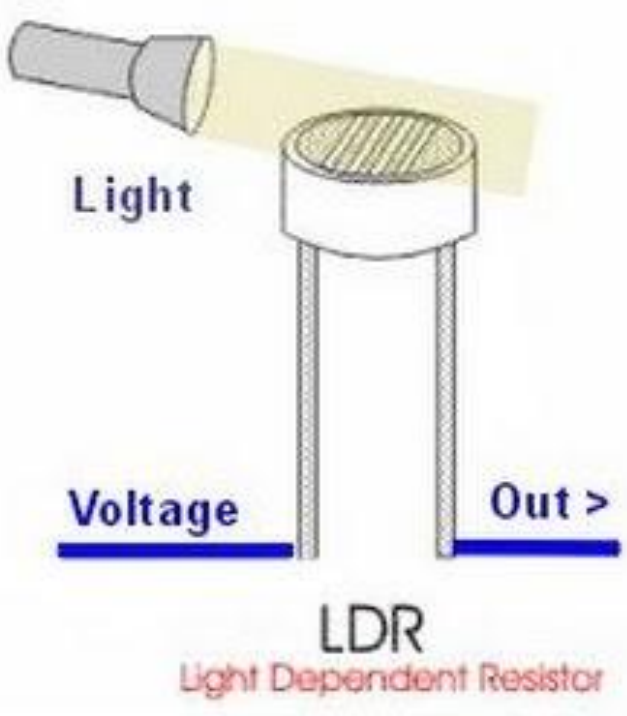
- Its resistance decreases with an increase in the intensity of light.

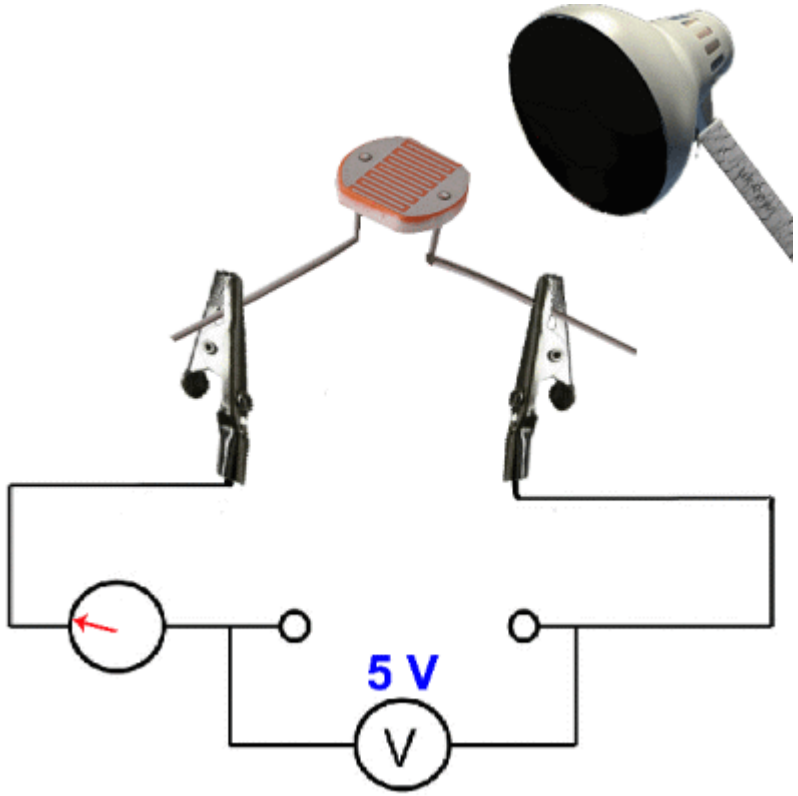
- It is often used as a
  - light sensor,
  - light meter,
  - Automatic street light,



Interfacing an Analog Sensor with Arduino UNO







- **When exhibited to light, the current through LDR increases**

Interfacing an Analog Sensor with Arduino UNO

Schematic Diagram

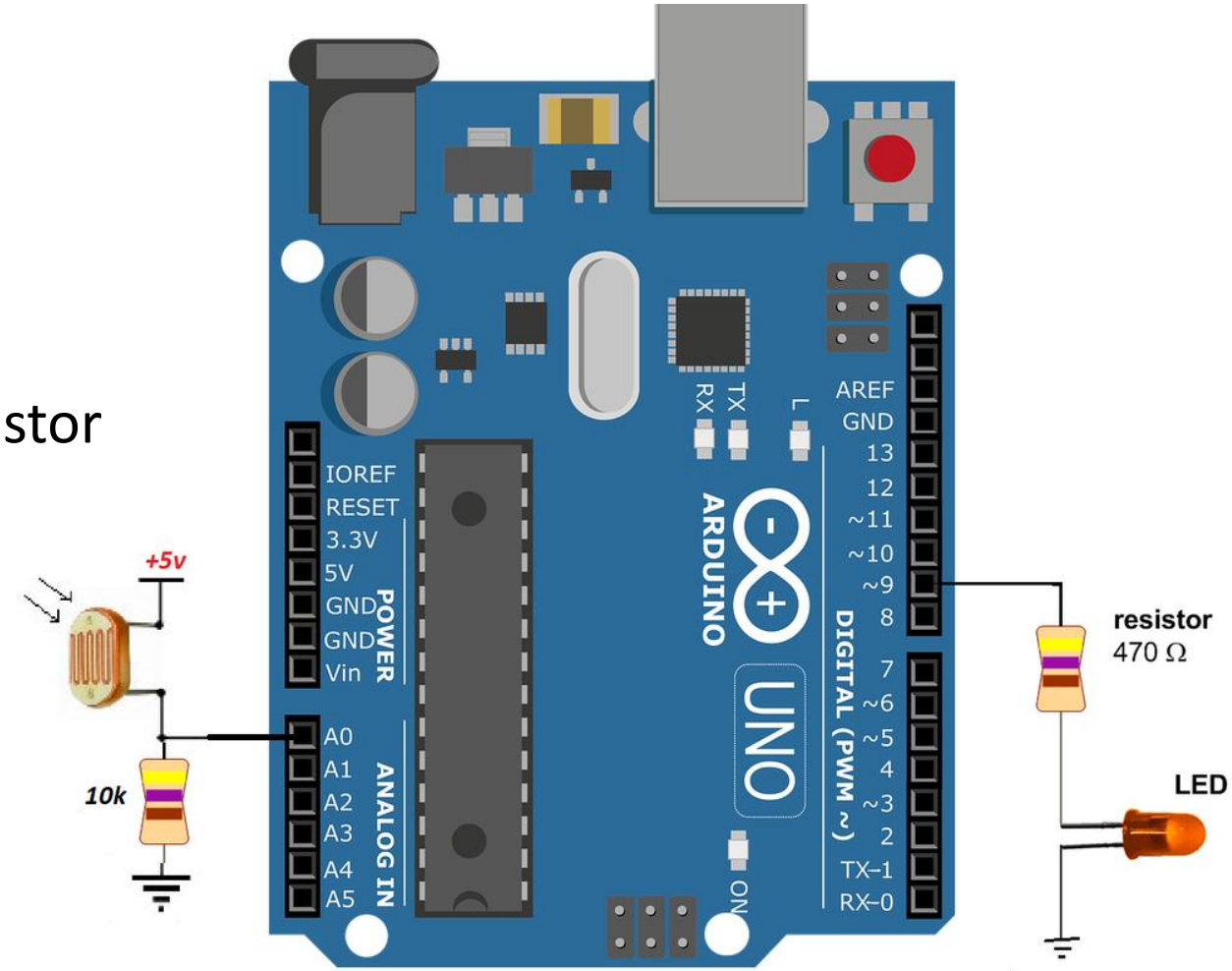
LDR
- one end to 5V
- another end to any analog pin

The same analog pin to GND- 10KΩ

LED

Long leg to any digital pin through 470Ω resistor

Short leg-to GND



Interfacing an Analog Sensor with Arduino UNO

Arduino Code

```
int ldrPin = A0;     // LDR connected to analog pin A0
int ledPin = 9;      // LED connected to digital pin 9
int ldrValue = 0;    // Variable to store LDR value

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);   // Initialize serial communication
}
```

Interfacing an Analog Sensor with Arduino UNO

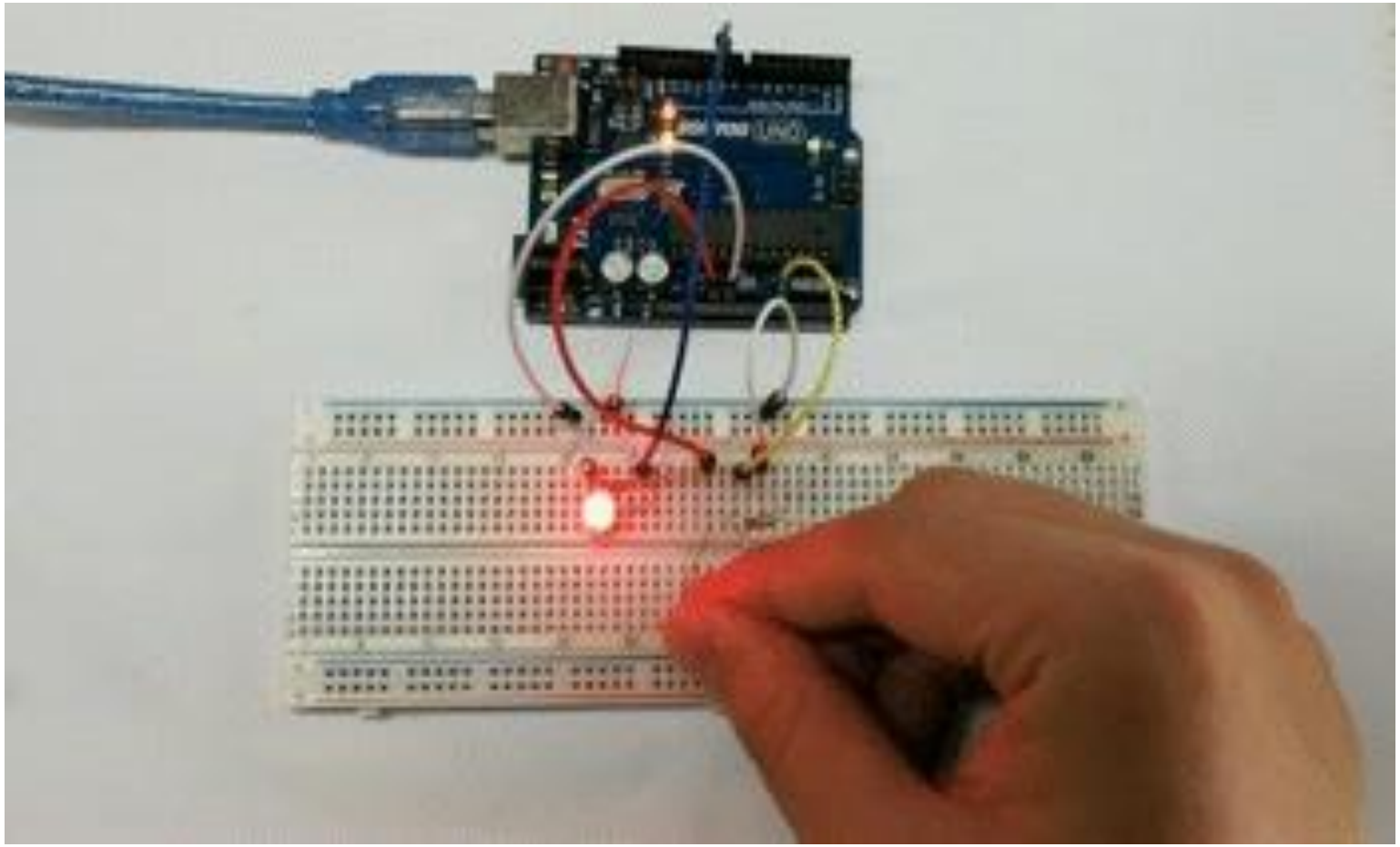
Arduino Code

```
void loop() {
  ldrValue = analogRead(ldrPin);   // Read LDR value
  Serial.print("LDR Value: ");
  Serial.println(ldrValue);

  if (ldrValue < 400) {
    digitalWrite(ledPin, HIGH); // If it is dark, turn ON LED
  } else {
    digitalWrite(ledPin, LOW);
  }
  delay(500);   // Delay for stability
}
```

Interfacing an Analog Sensor with Arduino UNO

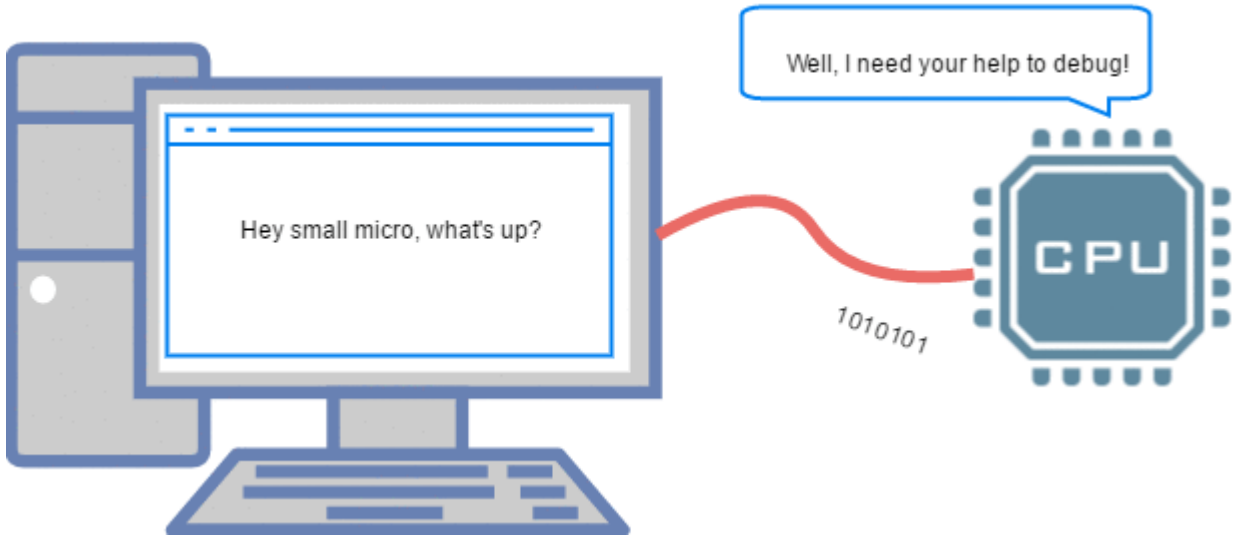
Expected Output

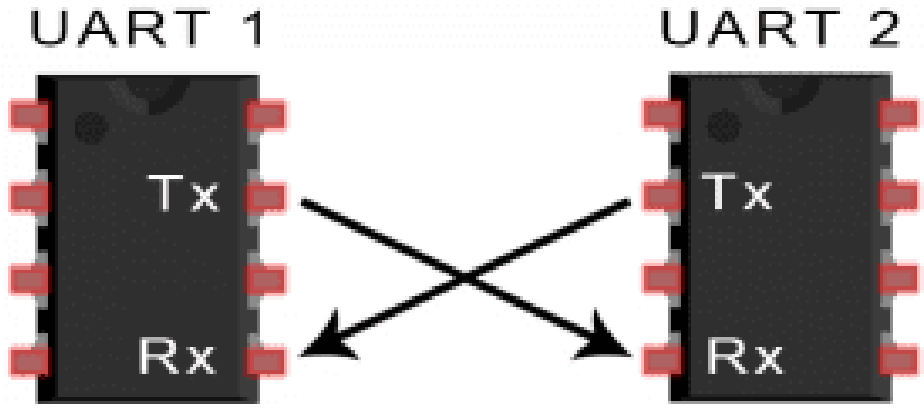


Demonstration of UART Communication Protocol

What Is UART?

- UART stands for Universal Asynchronous Receiver/Transmitter.
- It is a hardware device (or circuit) used for serial communication between two devices.
- Asynchronous refers that there is no clock signal involved in communication.



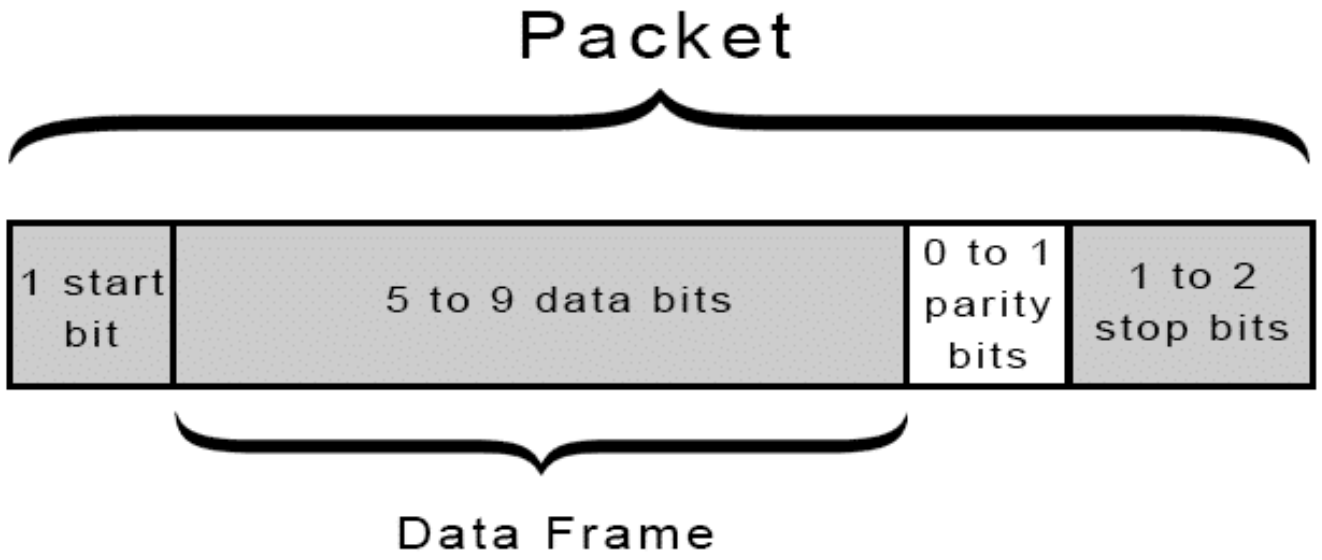


Demonstration of UART Communication Protocol

How Does UART Work?

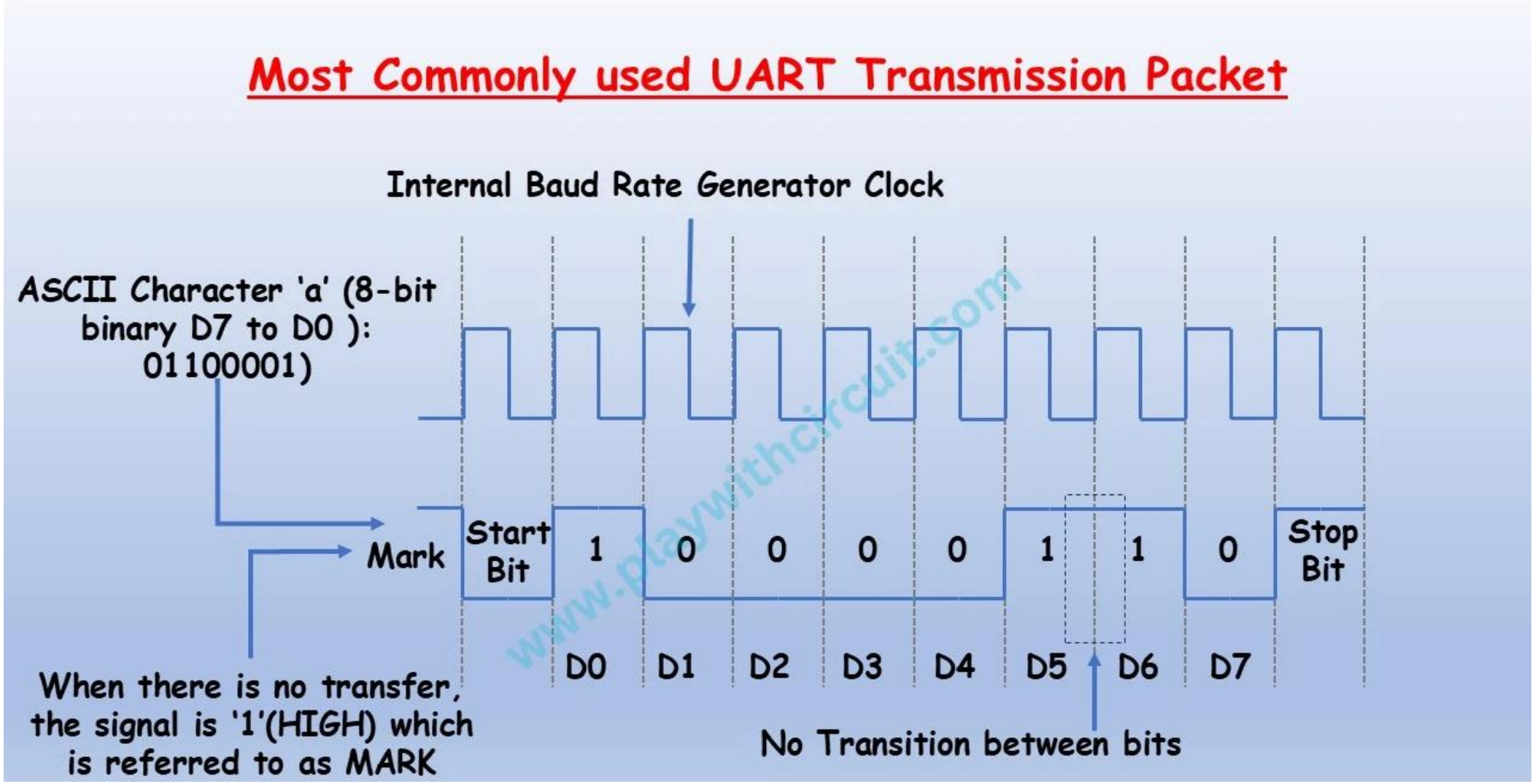
- UART works by converting data into **packets** for sending or **rebuilding data from packets received**.

UART Frame format



Demonstration of UART Communication Protocol

UART Frame format



Demonstration of UART Communication Protocol

UART Parameters

- UART has settings that need to be the **same on both devices** to have proper communication.
- These UART settings are the ***baud rate***, ***data length***, ***parity bit***, ***number of stop bits,*** **and** ***flow control***.

Demonstration of UART Communication Protocol

Baud Rate

- Baud rate is the number of *bauds per second* a UART device can transmit/receive.
- We need to set both UART devices with the same baud rate to have the proper transmission of data.
- Common values for baud rate are 9600, 1200, 2400, 4800 , 19200, 38400, 57600, and 115200 bps

DATA LENGTH

- Data length refers to the number of bits per byte of data.

Demonstration of UART Communication Protocol

PARITY BIT

The parity bit is a bit added to the transmitted data and tells the receiver if the number of 1's in the data transmitted is odd or even. The possible setting for Parity Bit is *Odd* or *Even*.

- **ODD** – the parity bit is '1' if there is an *odd* number of 1's in the data frame
- **EVEN** – the parity bit is '0' if there is an *even* number of 1's in the data frame

NUMBER OF STOP BITS

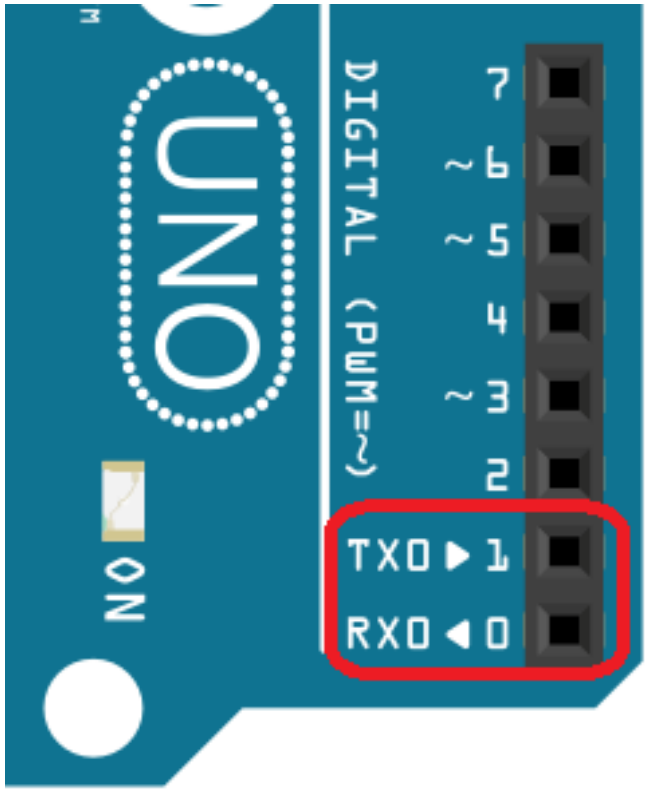
- UART devices can use *none*, *one* or *two* stop bits to mark the end of a set of bits (called packets) transmitted.

FLOW CONTROL

- Flow Control is the method to avoid the risk of losing data when transmitting data over UART. The UART device uses special characters as flow control to start/stop transmission.

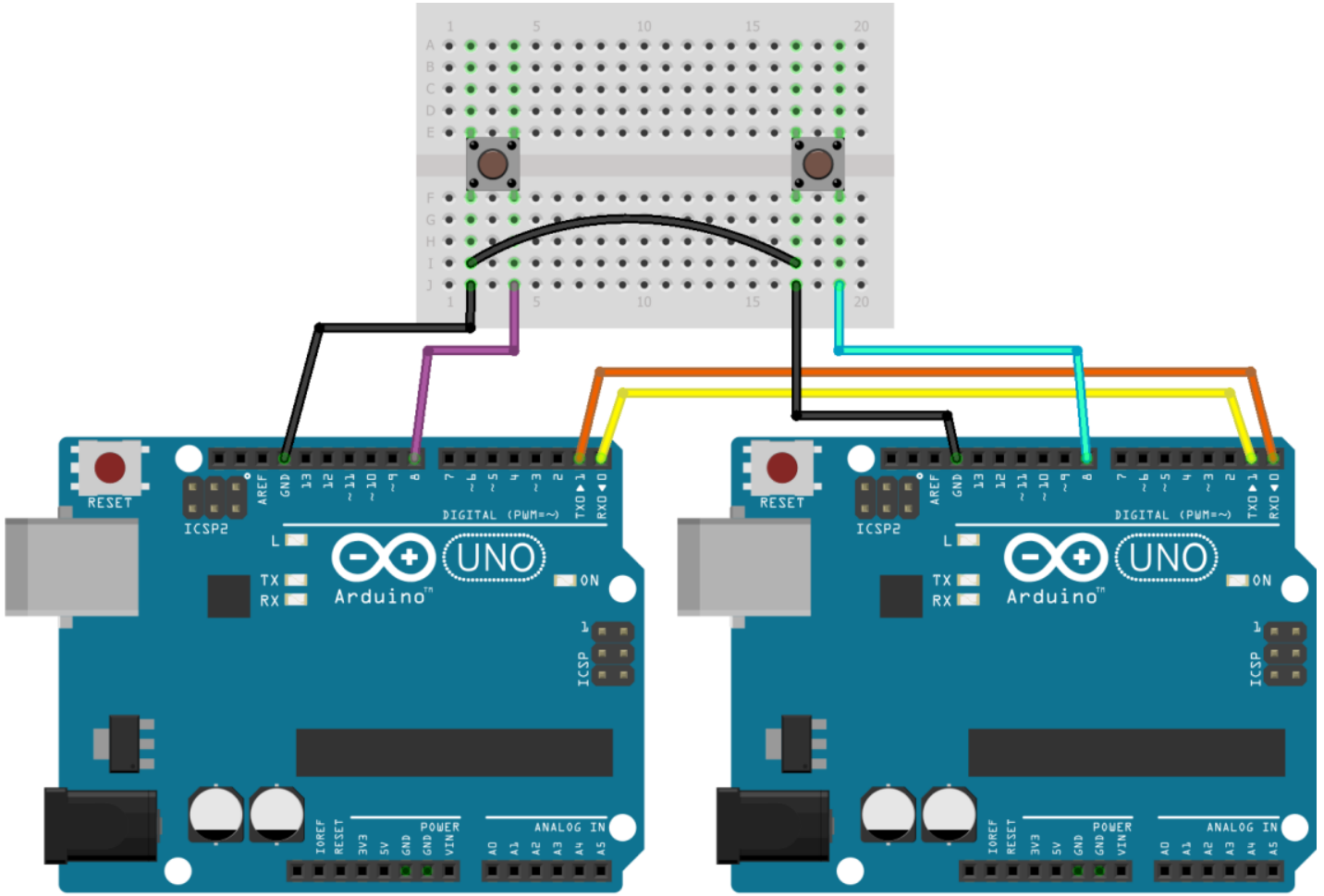
Demonstration of UART Communication Protocol

Arduino UART Interface



Demonstration of UART Communication Protocol

Schematic Diagram



Arduino code-UART

```
int button=8;
int led=13;
void setup() {
  pinMode(button, INPUT_PULLUP); // set push button
pin as input
  pinMode(led, OUTPUT);      // set LED pin as output
  Serial.begin(9600);       // initialize UART with baud
rate of 9600 bauds per second
}
```

Arduino code-UART (Receiver)

```
void loop() {
  if (Serial.available()) {

     char data_rcvd = Serial.read();    // read one byte from serial buffer and save to data_rcvd


  if (data_rcvd == '1') digitalWrite(led, HIGH); // switch LED On
     else digitalWrite(13, LOW);  // switch LED Off
   }
```

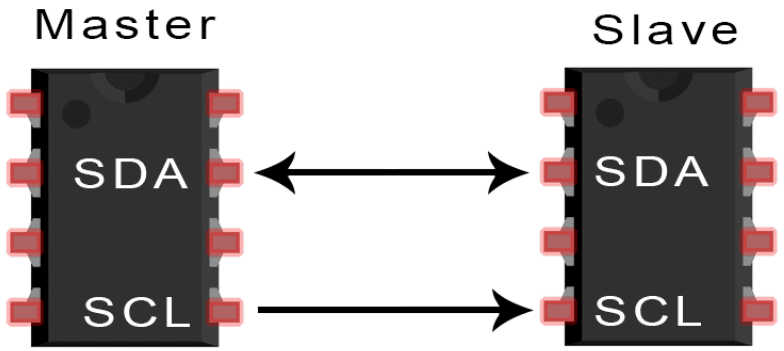
Arduino code-UART (Transmitter)

```
 if (digitalRead(button) == HIGH) Serial.write('0');
                    // send the char '0' to serial if button is not pressed.
  else Serial.write('1');
                    // send the char '1' to serial if button is pressed.
}
```

Demonstration of I2C Communication Protocol

What Is I2C?

- I2C or I^2C is short for Inter-Integrated Circuit, a synchronous serial communication protocol developed by Phillips for communication between a fast Microcontroller and relatively slow peripherals (like Memory or Sensors) using just two wires.
- Hence, it is sometimes also known as TWI (Two Wire Interface).
- Using I2C, you can transmit data at rates
  - 100 kbit/s (clock 100 kHz – Standard Mode),
  - 400 kbit/s (clock 400 kHz – Fast Mode),
  - 1 Mbit/s (clock 1 MHz – Fast Mode Plus) and
  - 3.4 Mbit/s (clock 3.4 MHz – High Speed Mode).

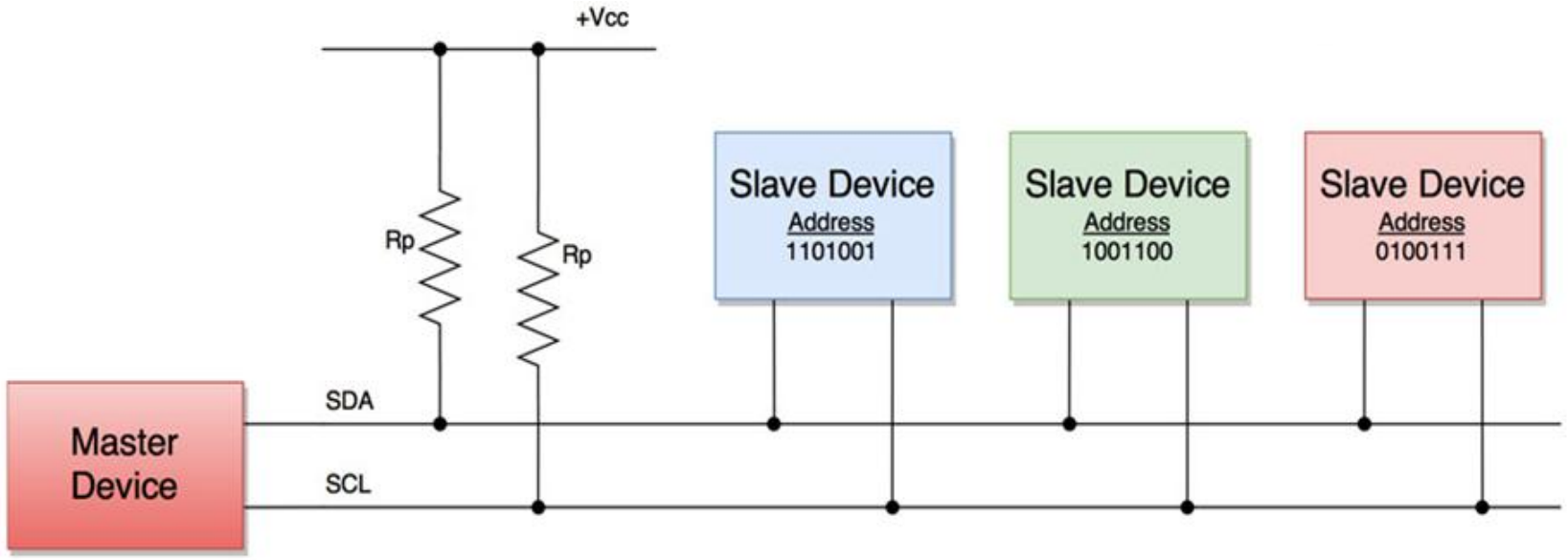


Demonstration of I2C Communication Protocol

I2C BUS

- The I2C Bus consists of two wires called the Serial Data (SDA) and the Serial Clock (SCL).
- Data is transmitted through the SDA line while the SCL line is used to synchronize the devices with the clock signal.

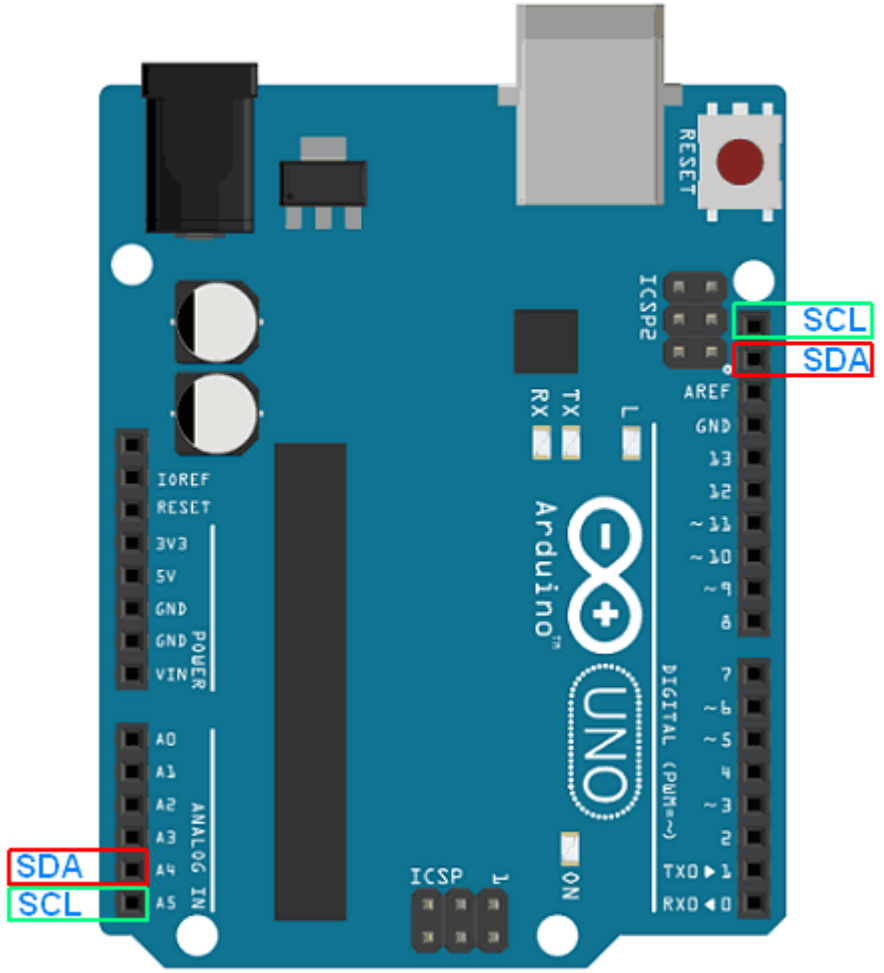
- Both these bus lines are open-drain drivers and hence you have to use pull-up resistors to keep them HIGH.



Demonstration of I2C Communication Protocol

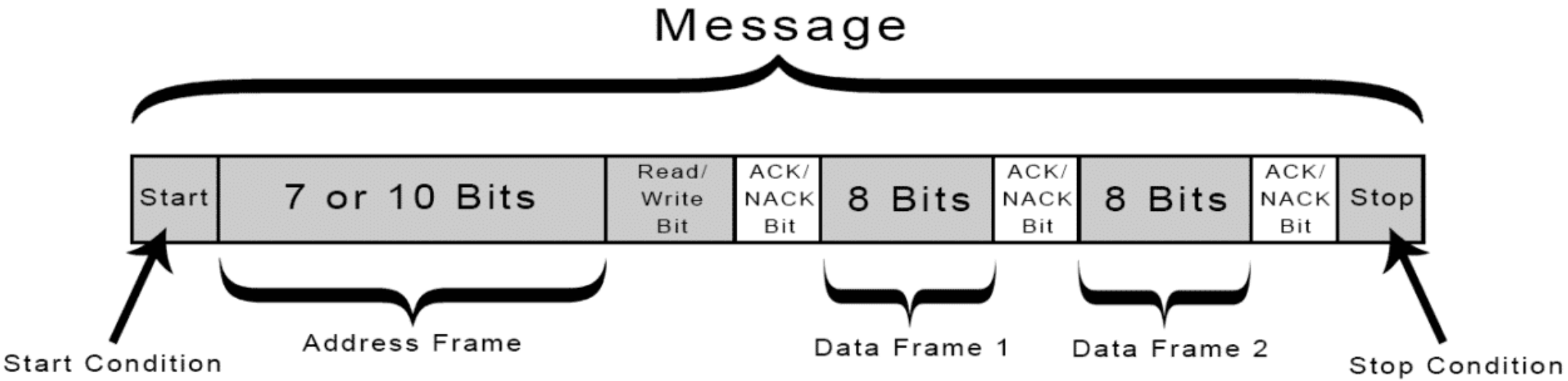
I2C in Arduino

- ATmega328P Microcontroller used in Arduino UNO and Nano supports I2C Data Transfer speeds up to 400 kHz.
- The A4 pin acts as SDA while the A5 pin acts as SCL.
- In R3 of original Arduino UNO, there are two more pins near Digital IO Pin 13 (near the USB socket), dedicated to SDA and SCL.



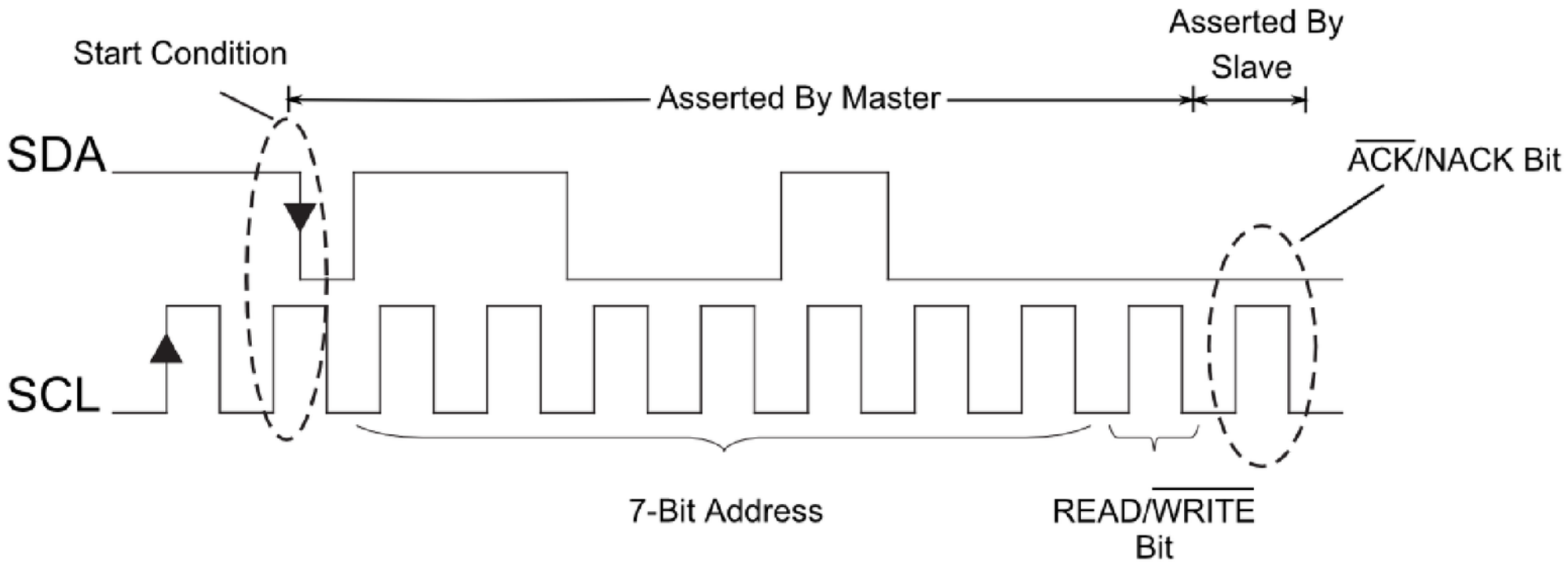
Demonstration of I2C Communication Protocol

I2C- Frame Format



Demonstration of I2C Communication Protocol

I2C- Frame Format



Demonstration of I2C Communication Protocol

I2C ADDRESSING

7-bit addressing

- The standard 7-bit addressing scheme provides 128 unique addresses (2^7)

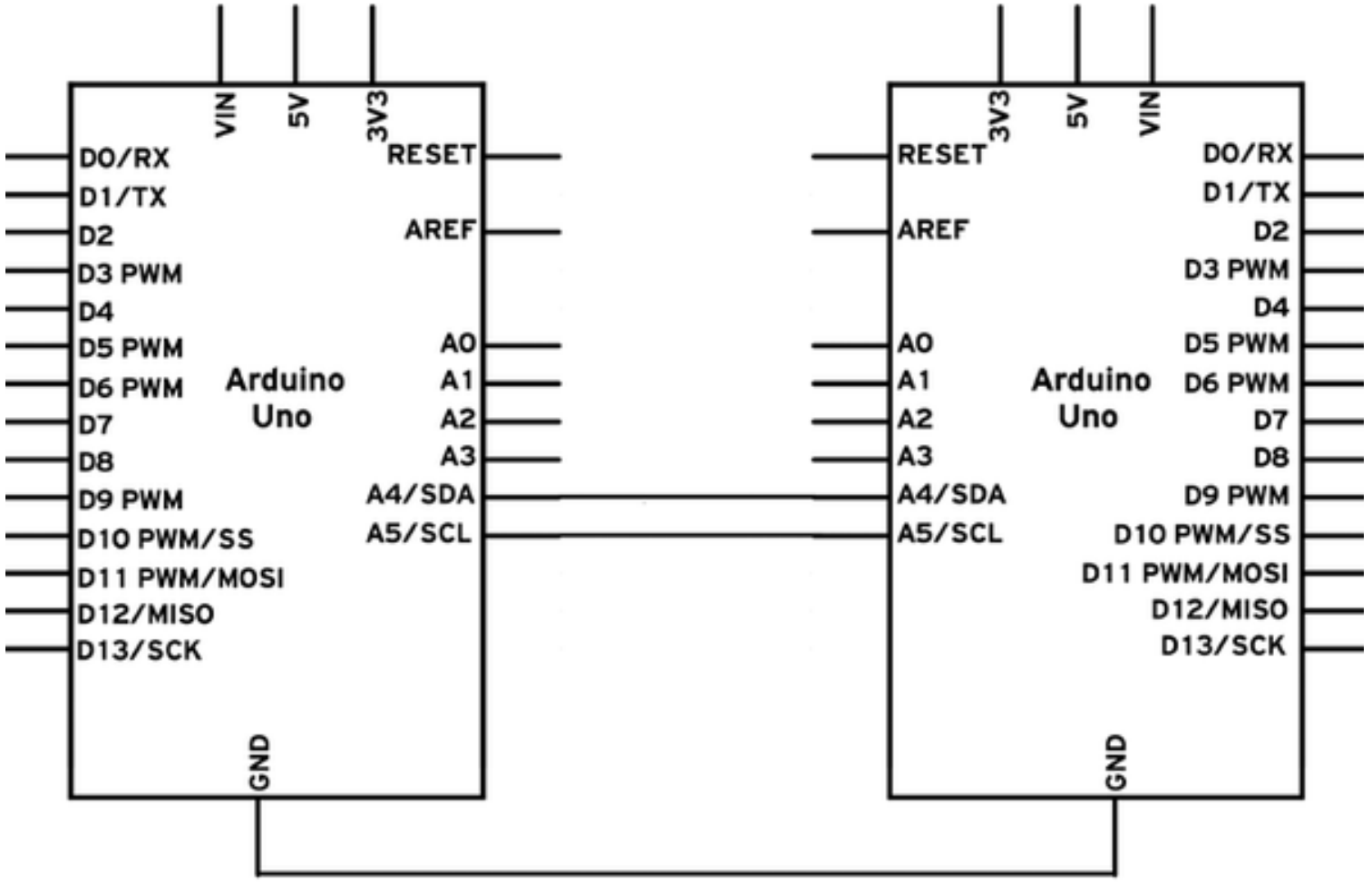
- However, 16 of these addresses are reserved for special purposes, leaving **112 usable addresses**.

10-bit addressing

1024 unique addresses (2^10)

Demonstration of I2C Communication Protocol

Schematic Diagram



Arduino code-I2C-Master

```
#include<Wire.h>
int  x = 0;
int button=7;
void setup() {
  // Start the I2C Bus as Master
  Wire.begin();
  Serial.begin(9600);
  pinMode(button,INPUT);
}
void loop() {
  Wire.beginTransmission(9); // transmit to device #9
  Wire.write(x);              // sends x
  Wire.endTransmission();    // stop  transmitting
  int val=digitalRead(button);
  if (val==HIGH)
  x++; // Increment x
  if (x > 5) x = 0; // `reset x once it  gets 6
  Serial.println(x);
  delay(500);
}
```

Arduino code-I2C-Slave

```
#include <Wire.h>
int   LED = 13;
int x = 0;
void setup() {
 // Define the LED pin as Output
  pinMode (LED, OUTPUT);
   Serial.begin(9600);
  // Start the I2C Bus as Slave on address 9
  Wire.begin(9);
  // Attach a function to trigger when something is received.
  Wire.onReceive(receiveEvent);
}
void   receiveEvent(int bytes) {
  x = Wire.read();    // read one character from the   I2C
}
```

Arduino code-I2C-Slave

```
void loop() {

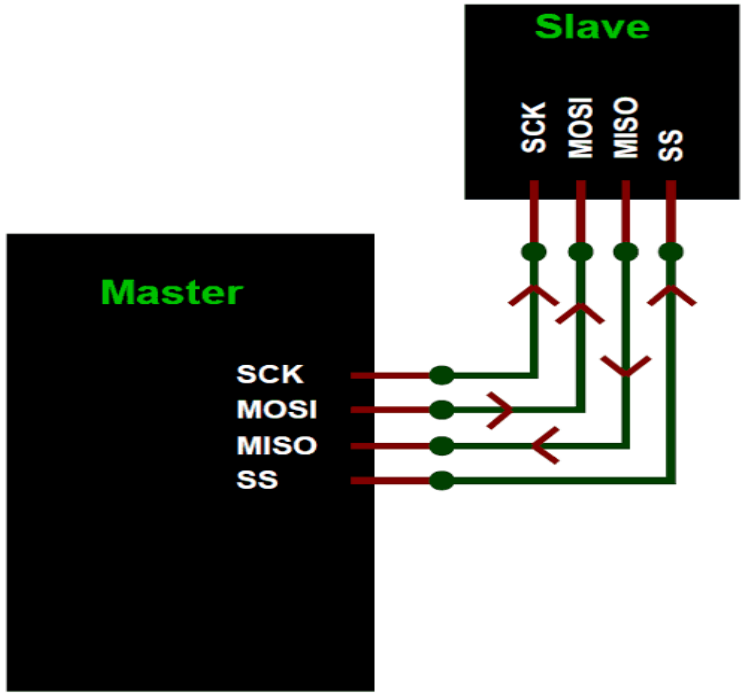
  Serial.println(x);
 // If value received is 3 blink LED for 400   ms
  if (x == 3) {
    digitalWrite(LED, HIGH);
    delay(400);}
    else{
   digitalWrite(LED,   LOW);
   delay(400);

 }
}
```

Demonstration of SPI Communication Protocol

What Is SPI?

- **SPI (Serial Peripheral Interface)** is a serial communication protocol.
- SPI interface was found by Motorola in 1970.
- SPI has a full-duplex connection, which means that the data is sent and received simultaneously.
- SPI is synchronous serial communication means the clock is required for communication purposes.



Demonstration of SPI Communication Protocol

SPI

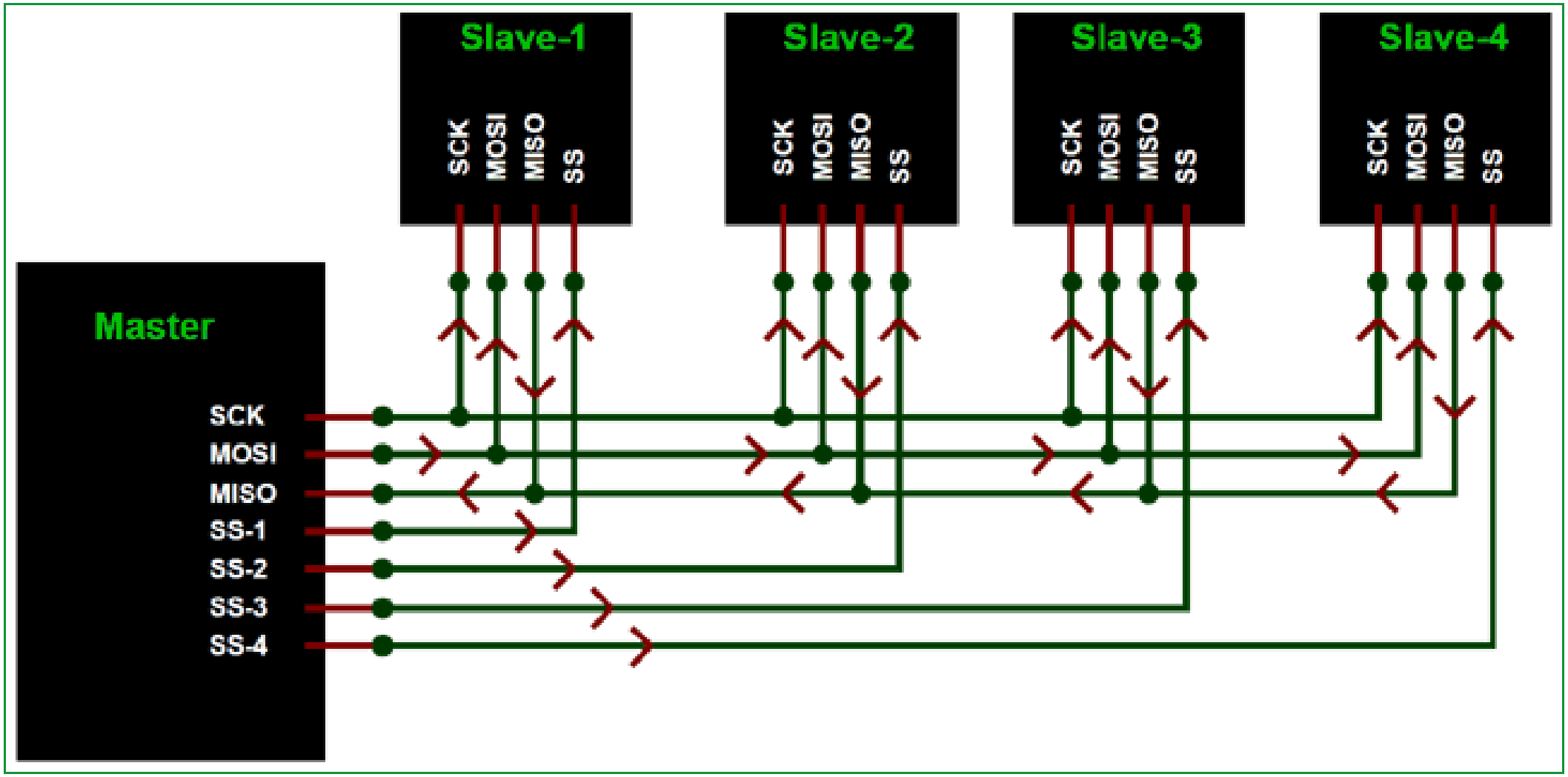
- A SPI has a master/Slave communication by using four lines.
- A SPI can have only one master and can have multiple slaves.
- A master is usually a microcontroller and the slaves can be a microcontroller, sensors, ADC, DAC, LCD etc.

SPI-pins

- **MISO (Master in Slave Out)** - The Slave line for sending data to the master.
- **MOSI (Master Out Slave In)** - The Master line for sending data to the peripherals.
- **SCK (Serial Clock)** - The clock pulses which synchronize data transmission generated by the master.
- **SS (Slave Select)** –Master can use this pin to enable and disable specific devices.

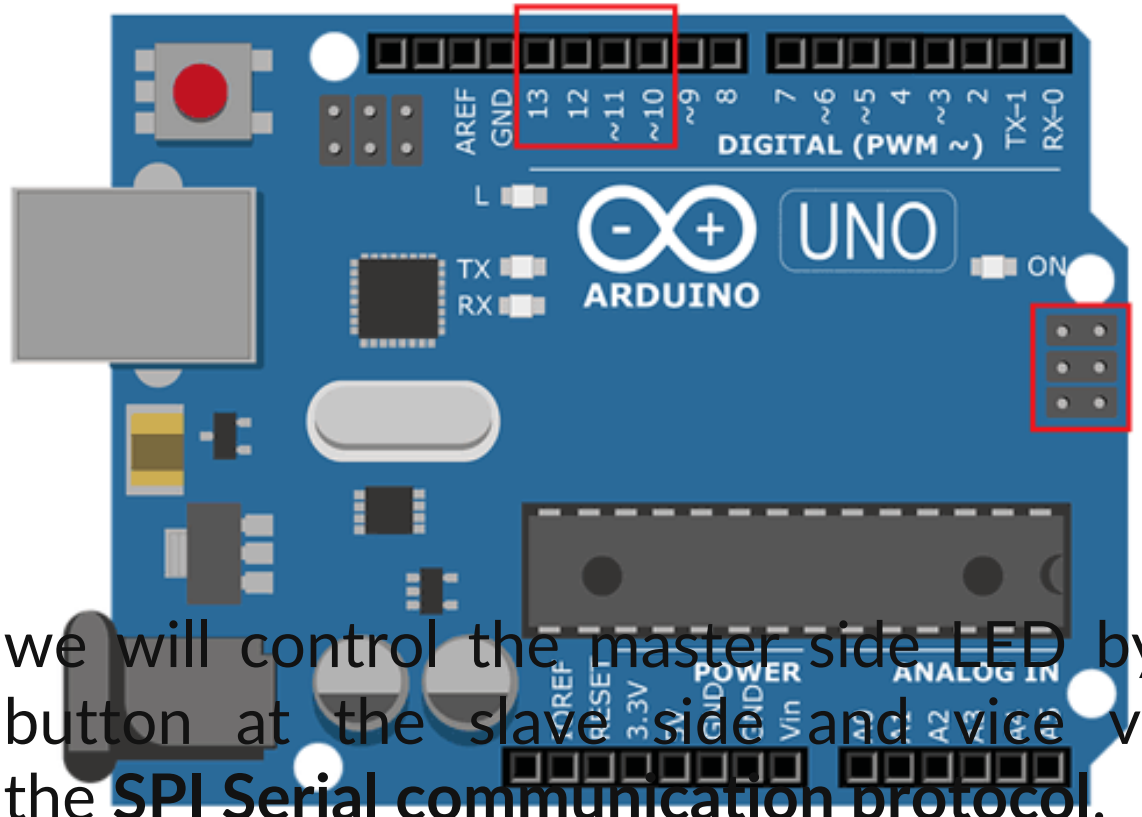
Demonstration of SPI Communication Protocol

SPI-multiple slaves



Demonstration of SPI Communication Protocol

SPI Pins in Arduino UNO



SPI Line	Pin in Arduino
MOSI	11 or ICSP-4
MISO	12 or ICSP-1
SCK	11 or ICSP-3
SS	10

- we will control the master side LED by the push button at the slave side and vice versa using the **SPI Serial communication protocol**.

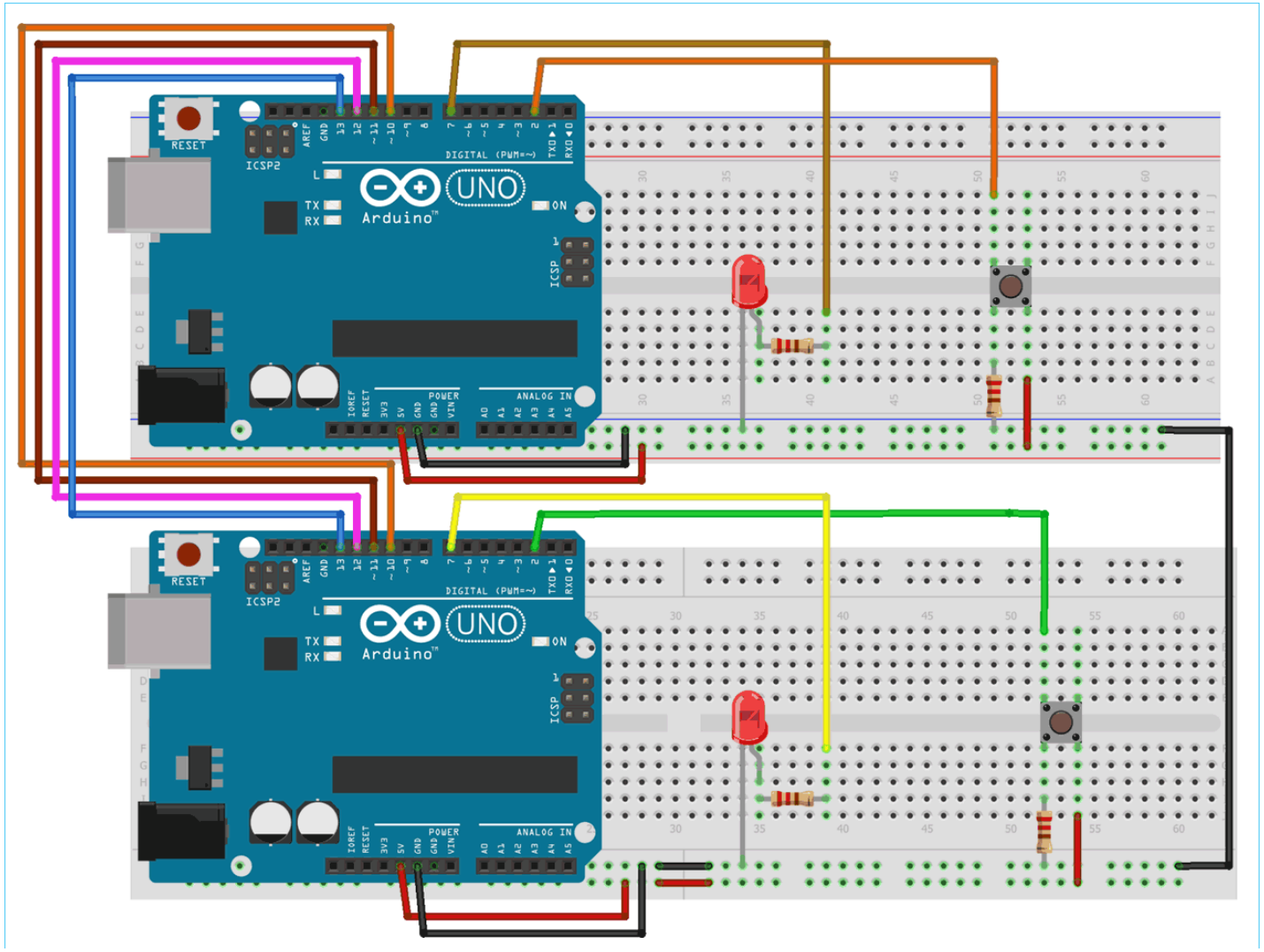
Demonstration of SPI Communication Protocol

What we do in our lab to demonstrate SPI

- we will control the master side LED by the push button at the slave side and vice versa using the **SPI Serial communication protocol**.

Demonstration of SPI Communication Protocol

Schematic Diagram



Arduino code-SPI-Master

```
#include<SPI.h>                    //Library for SPI

#define LED 7

#define ipbutton 2

int buttonvalue;

int x;
```

Arduino code-SPI-Master

```
void setup (void)
{
  Serial.begin(115200);                  //Starts Serial Communication at Baud Rate 115200
   pinMode(ipbutton,INPUT);               //Sets pin 2 as input
   pinMode(LED,OUTPUT);                    //Sets pin 7 as Output
   SPI.begin();                          //Begins the SPI commnuication
   SPI.setClockDivider(SPI_CLOCK_DIV8);  //Sets clock for SPI communication at 8 (16/8=2Mhz)
   digitalWrite(SS,HIGH);  //Setting SlaveSelect as HIGH (So master doesnt connnect with slave)
}
```

Arduino code-SPI-Master

```
void loop(void)
{
 byte Mastersend,Mastereceive;
 buttonvalue = digitalRead(ipbutton);   //Reads the status of the pin 2
 if(buttonvalue == HIGH) //Logic for Setting x value (To be sent to slave) depending upon input from pin 2
 {
  x = 1;
 }
 else
 {
  x = 0;
 }
 digitalWrite(SS, LOW);                //Starts communication with Slave connected to master
 Mastersend = x;
```

Arduino code-SPI-Master

```
Mastereceive=SPI.transfer(Mastersend); //Send the mastersend value to slave also receives value from slave
  if(Mastereceive == 1)                //Logic for setting the LED output depending upon value received from slave
  {
    digitalWrite(LED,HIGH);            //Sets pin 7 HIGH
    Serial.println("Master LED ON");
  }
  else
  {
   digitalWrite(LED,LOW);              //Sets pin 7 LOW
   Serial.println("Master LED OFF");
  }
  delay(1000);
}
```

Arduino code-SPI-Slave

```
#include<SPI.h>
#define LEDpin 7
#define buttonpin 2
volatile boolean received;
volatile byte Slavereceived,Slavesend;
int buttonvalue;
int x;
```

Arduino code-SPI-Slave

```
void setup()
{
 Serial.begin(115200);
pinMode(buttonpin,INPUT);              // Setting pin 2 as INPUT
pinMode(LEDpin,OUTPUT);                 // Setting pin 7 as OUTPUT
pinMode(MISO,OUTPUT);                   //Sets MISO as OUTPUT (Have to Send data to Master IN
SPCR |= _BV(SPE);                     //Turn on SPI in Slave Mode
received = false;
SPI.attachInterrupt();                //Interuupt ON is set for SPI commnucation
}
ISR (SPI_STC_vect)                     //Inerrrput routine function
{
Slavereceived = SPDR;          // Value received from master if store in variable slavereceived
received = true;                    //Sets received as True
}
```

Arduino code-SPI-Slave

```
void loop()
{
if(received)              //Logic to SET LED ON OR OFF depending upon the value recerived from master
   {
   if (Slavereceived==1)
    {
    digitalWrite(LEDpin,HIGH);          //Sets pin 7 as HIGH LED ON
    Serial.println("Slave LED ON");
   }
else
    {
      digitalWrite(LEDpin,LOW);           //Sets pin 7 as LOW LED OFF
      Serial.println("Slave LED OFF");
     }
      buttonvalue = digitalRead(buttonpin);  // Reads the status of the pin 2
```

Arduino code-SPI-Slave

```
if (buttonvalue == HIGH)              //Logic to set the value of x to send to master
     {
       x=1;
      }
else
  {
     x=0;
     }
  Slavesend=x;
  SPDR = Slavesend;                        //Sends the x value to master via SPDR
  delay(1000);
}
}
```

IoT Device Interoperability : *Challenges and Solutions*

1. What Is Interoperability?
2. Four Core Challenges
3. Standardization Efforts
4. Middleware & Semantic Solutions
5. Open APIs, Cloud & AI Approaches
6. Key Takeaways & Discussion

What Is IoT Interoperability?

The ability of heterogeneous devices, platforms, and applications to exchange data and use it meaningfully — without custom, one-off integration for every pair of systems.

T Technical

Devices connect over a shared physical & network layer (radio, transport).

S Syntactic

Exchanged data follows a common structure or format (JSON, XML).

S Semantic

Shared meaning: both sides interpret the data identically.

O Organizational

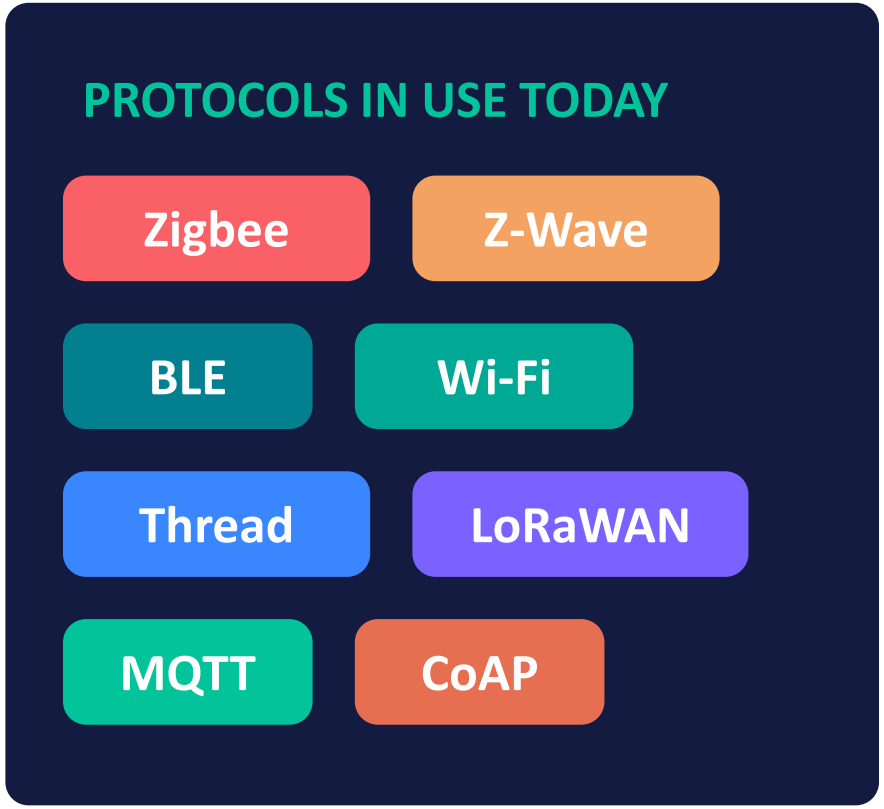
Aligned business processes, policies, and governance.

CHALLENGE 1 OF 4

Protocol & Standard Fragmentation

1

- Devices communicate over many incompatible radio and network protocols.
- No single universal transport or application-layer standard exists.
- Gateways and translators are required to bridge separate stacks.

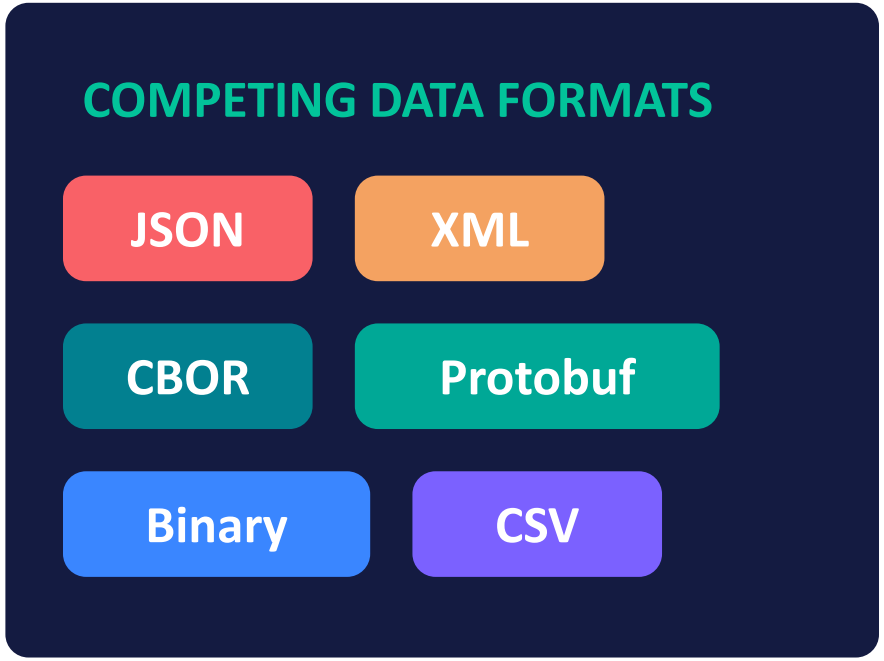


CHALLENGE 2 OF 4

Data & Semantic Heterogeneity

2

- The same reading is encoded in different formats across vendors.
- Field names are ambiguous — one label can mean different things.
- There is no shared vocabulary describing what the data represents.



CHALLENGE 3 OF 4

Security & Vendor Lock-in

3

- Proprietary ecosystems intentionally restrict cross-vendor communication.
- Authentication and encryption practices vary widely between platforms.
- Closed APIs discourage independent, third-party integration.



CHALLENGE 4 OF 4

Scalability & Legacy Integration

4

- Networks must support huge numbers of constrained, low-power devices.
- Legacy equipment was never designed for modern connectivity.
- Firmware updates and device lifecycles are difficult to standardize.



SOLUTION 1 OF 3

Standardization & Unified Protocols

1

Matter (Connectivity Standards Alliance)

One application layer running over Wi-Fi, Thread, and Bluetooth LE.

2

oneM2M & OCF Frameworks

Common service layers so devices from different makers share one model.

3

MQTT & CoAP Messaging

Lightweight, widely adopted protocols built for constrained devices.

SOLUTION 2 OF 3

Middleware, Gateways & Semantic Web

1

Interoperability Middleware

Translation layers convert between protocols in real time.

2

Ontologies & Semantic Tagging

Shared vocabularies (e.g. SSN/SOSA, JSON-LD) attach real meaning to data.

3

Edge Gateways

Normalize and clean device data before it ever reaches the cloud.

SOLUTION 3 OF 3

Open APIs, Cloud Platforms & AI

1

Open, RESTful APIs

Publish clear, documented interfaces that invite third-party integration.

2

Cloud Interoperability Platforms

Managed IoT clouds unify device onboarding, control, and monitoring.

3

AI-Assisted Schema Mapping

Machine learning automates translation between mismatched data schemas.

SYNTHESIS

Mapping Challenges to Solutions

CHALLENGE	SOLUTION APPROACH
Protocol Fragmentation	Matter, oneM2M, OCF, MQTT/CoAP
Data & Semantic Gaps	Ontologies, JSON-LD, semantic middleware
Security & Lock-in	Open APIs, shared auth standards
Scalability & Legacy	Edge gateways, cloud device management